Smart Contract Assessment

Moonwell

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Oct 4, 2026 - Oct 14, 2026

SEVERITY BREAKDOWN

53	39	3	11
All Findings	Solved	Risk Accepted	Acknowledged



INTRODUCTION

`Moonwell` engaged `Halborn` to perform a security assessment of their smart contracts from October 5th, 2026 to October 14th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The `Moonwell` codebase in scope consists of smart contracts implementing per user Stock Accounts that hold a basket of tokenized stocks and USDC rebalanced through CoW Swap orders validated against Aerodrome TWAP and Chainlink prices, together with a reward drop system that funds MultiRewards streams and lets MAMO stakers compound or reinvest those rewards.

ASSESSMENT SUMMARY

`Halborn` was allocated 8 days for this engagement and assigned 2 full-time security engineers to conduct a comprehensive review of the smart contracts within scope. The engineers are experts in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which were partially addressed by the `Moonwell` team. The main recommendations were:

- `Add a caller supplied minimum output and deadline to StockAccountStrategy.withdrawAll() and StockAccountStrategy.withdraw(), forward that deadline to the router, and keep the TWAP based per leg floor as a second check.`
- `Introduce a registry minimum sell value so that StockAccountStrategy._sellable() skips balances below it and StockAccountStrategy._planSells() plans every sell leg at either zero or at least that value.`
- `Size every cosigned order that sells a full token balance to at most that balance minus a margin covering the fee that StockAccountStrategy.payFees() can take in that token before the order expires.`

TEST APPROACH AND METHODOLOGY

`Halborn` conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.
The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (`Foundry`).
- Fork testing against main networks (`Foundry`).

- Static security analysis of scoped contracts, and imported functions (`Slither`).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational

9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

URLS

(a) URL:
https://github.com/moonwell-fi/mamo-contracts/pull/113/changes/BASE..f0dbde357734f4b021551639c3d03ba6cecf19b0

(b) Items in scope:

- src 3 files
  - DropAutomationV2.sol Solidity
  - MamoStakingRegistryV2.sol Solidity
  - MamoStakingStrategyV2.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

(a) URL:
https://github.com/moonwell-fi/mamo-contracts/pull/96/changes/BASE..84d8e57047d21f1fef464f32e2dccea931f1e281

(b) Items in scope:

- src 4 files
  - StockAccountPriceChecker.sol Solidity
  - StockAccountRegistry.sol Solidity
  - StockAccountStrategy.sol Solidity
  - StockAccountStrategyFactory.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- 4492632a73f3f0b76d781d2ab569b3334d860b5d
- 73631187bb044169bf07c7d862a853a095219226
- a3db6d56e3a4faf62efe51a66799463000d4efc96
- 7192df285e75f192c140a282089bd72c0933ec3c
- 1e15f147d5fd83f2efc039fc01da47b4e5162b94
- b2d9bcc0e4cd6b5271d0362edd47fd3115e80455
- 12c31c4bc99d2df02850b82793d46cbc622eccff

- 392dac8da5850Ob121a9ae67d1a6377bdd862b51
- 05b3af25d4fd68dab0826001f15b093ac0f24f89
- 2a90fd02791081b43e8732bf396c365ed2388836
- 719bd53fa8cfc56e639c04e4cbbd2971974a81a6
- c191a8ef18f949ab883bbd8d5e2187d85b99737e
- 78e818469f03ce9eb0f80ce8812b3b67bf134f1e
- 2567ad77fcd31d5c30d77b1107fc5dcd7c163216
- f845823db16d35a8de9f484e5683106392ff0fa3
- 8ce6b395e86d9355ee81a5707942e6fa3404d6a6
- 4ef8dea05b5e469a26d325a8ee362b0474b67157
- 1fd727b54c4f1ff6afbea39bb950671c360f5189
- 705bbf7222ec91e179e83b9420e538eb793d0772
- 87b14a50146eebaf8bd1cb812e0774e3fb96d92e
- 406ec7247550a340db0cfb5c879a164016a491cc
- 29f804a8915143b6e6a727074e6894a2a22aba67
- b3dec9728fc2a981de1b5276cdfa9f175807db73
- c6b9fd887606c18ca75c0bc9ab888d736dfde7d0
- f8de526fcb0137f2592072a3570f50318ca80e24
- aafd8b8157cc454c81ac051bac04aada64a0090e
- 82de9ef80061c40959d434ad5940be563adee22b
- 2c9a6a84f32a85f9a2115084d812befc656657cc
- 93a2b40d79b82c02a5f52a07d40312ae0289b73e
- 45196ba66deb6c61ef09258e4579300ceaab211c
- 7cf41eaa282b0d03cc7b21e545b13cef51580349
- 9c9b2ad595ee831ae5d840ef6f7106bfc6445275
- 42e2176983f4f2a24cd847fdba6bad991c89b588
- e8fb462deb458219b0cdb0933435ebab5a6cd43a
- 3b08f6a1e34867eb608c5e1d7c59bf3839dfa4a4
- fb8d8fa3e8b5f4020519e896f2f8f548a7b4614e
- d25548ad51c6814b486cb382e96af2a0193cc08b
- 6a8fe389127db5bbcbc88b88f4407adb68c2b8d2

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Owner cash-exit floor uses only the in-transaction TWAP, with no caller minimum or deadline	Low	3.4	SOLVED 10/15/2026
HAL-002	A dust balance of a listed token makes withdraw and withdrawAll revert	Low	2.5	SOLVED 10/15/2026
HAL-003	Permissionless payFees in the sell token makes full-balance orders unfillable	Low	2.5	RISK ACCEPTED
HAL-004	Upgrade does not migrate the stored principal	Low	2.5	SOLVED 10/15/2026
HAL-005	Manipulable stock reward swap floor due to same pool TWAP reference	Low	2.2	RISK ACCEPTED
HAL-006	Registry administrator can replace the price checker or router and loosen limits for every account at once	Low	2.0	SOLVED 10/15/2026
HAL-007	Centralization risks	Low	2.0	RISK ACCEPTED
HAL-008	No sequencer uptime check on the TWAP and cbBTC Chainlink price paths	Informational	1.9	SOLVED 10/15/2026
HAL-009	Any member of the shared backend role can set the fee recipient of every account	Informational	1.5	SOLVED 10/15/2026
HAL-010	payFees caller chooses the fee token at a manipulable TWAP	Informational	1.1	ACKNOWLEDGED
HAL-011	Halted stock rewards still sold due to missing status check in compound	Informational	1.1	SOLVED 10/15/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-012	Account value that excludes halted or unpriceable holdings feeds the range check and the fee, and a zero value resets the fee clock	Informational	1.0	SOLVED 10/15/2026
HAL-013	Single-step ownership transfer process	Informational	1.0	ACKNOWLEDGED
HAL-014	cbBTC valued with the BTC/USD feed	Informational	0.9	ACKNOWLEDGED
HAL-015	Deposit cap limits tracked principal, not account value; full in-kind exit resets principal when a transfer is skipped	Informational	0.8	SOLVED 10/15/2026
HAL-016	Reward dust left unstreamed due to rounding on notifications during an active period	Informational	0.8	SOLVED 10/15/2026
HAL-017	Stock drop deployment does not require the sequencer-guarded price checker before V2 compounding goes live	Informational	0.8	SOLVED 10/15/2026
HAL-018	Order-signer key has no cumulative, rate, count or direction limit	Informational	0.7	SOLVED 10/15/2026
HAL-019	TWAP pool accepted without liquidity or observation checks	Informational	0.6	SOLVED 10/16/2026
HAL-020	Pinned factory implementation and owner-only upgrades halt rollout	Informational	0.5	SOLVED 10/15/2026
HAL-021	Selling down a SellOnly token stops at the ceilings of the receiving legs	Informational	0.5	SOLVED 10/15/2026
HAL-022	Missing input validation	Informational	0.5	SOLVED

#	TITLE	SEVERITY	SCORE	STATUS
				10/15/2026
HAL-023	_reducePrincipal pricing differs from NAV for Halted and unpriceable tokens	Informational	0.4	SOLVED 10/15/2026
HAL-024	withdrawToken and recoverERC20 revert when the fee consumes the exit token	Informational	0.4	SOLVED 10/15/2026
HAL-025	Rewards notified while no MAMO is staked are not distributed	Informational	0.4	ACKNOWLEDGED
HAL-026	Deprecated staking factory keeps BACKEND_ROLE after the stock drop cutover	Informational	0.4	SOLVED 10/15/2026
HAL-027	Stock hop floor priced on a different pool after a pricing pool change	Informational	0.4	SOLVED 10/15/2026
HAL-028	Ineffective on-chain slippage floor due to same-transaction pool quote	Informational	0.3	ACKNOWLEDGED
HAL-029	Same-batch orders validated against the same pre-trade state	Informational	0.3	ACKNOWLEDGED
HAL-030	Account ownership transfer desyncs factory per-user creation state	Informational	0.3	SOLVED 10/15/2026
HAL-031	Slippage and deviation caps accept zero	Informational	0.3	SOLVED 10/15/2026
HAL-032	Unguarded fee transfer lets a blocked fee recipient stop cash exits and deposits	Informational	0.2	ACKNOWLEDGED
HAL-033	Reverting balanceOf of a listed token blocks valuation	Informational	0.2	SOLVED 10/15/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-034	Fee post-hook and valuation gas grow with the listed-token count, which has no bound	Informational	0.2	SOLVED 10/16/2026
HAL-035	Pricing pool is not checked against the router's factory	Informational	0.2	SOLVED 10/15/2026
HAL-036	maxTimePriceValid returns the TWAP window for unlisted tokens	Informational	0.2	SOLVED 10/15/2026
HAL-037	setPriceChecker probe covers only the token-to-asset direction	Informational	0.2	SOLVED 10/15/2026
HAL-038	No cross-check of the factory, registry and price checker assets	Informational	0.2	SOLVED 10/15/2026
HAL-039	Cached CoW domain separator and relayer not validated and never re-synced	Informational	0.2	SOLVED 10/15/2026
HAL-040	TWAP window accepts any nonzero value	Informational	0.2	SOLVED 10/15/2026
HAL-041	Range check uses the signed buy amount and checks only the two trade legs	Informational	0.2	ACKNOWLEDGED
HAL-042	Management fee computed on a mid-settlement NAV	Informational	0.2	ACKNOWLEDGED
HAL-043	Reinvest batch reverts due to unhandled MultiMarket deposit failure	Informational	0.2	SOLVED 10/15/2026
HAL-044	Reward tokens listed in MamoStakingRegistryV2 without an interface check or a list bound	Informational	0.2	SOLVED 10/15/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-045	Backend can create an account with a basket and cash target that the user did not approve	Informational	0.1	ACKNOWLEDGED
HAL-046	Reward token pool not validated against the token in MamoStakingRegistryV2	Informational	0.1	SOLVED 10/15/2026
HAL-047	withdrawGauge transfers the requested amount instead of the amount the gauge returned	Informational	0.1	SOLVED 10/15/2026
HAL-048	Staking accounts can be created with a mismatched token configuration	Informational	0.1	SOLVED 10/15/2026
HAL-049	Gauge rewards left unclaimed due to removal without a final claim	Informational	0.1	SOLVED 10/15/2026
HAL-050	Reinvest blocked while MultiRewards is paused	Informational	0.1	SOLVED 10/15/2026
HAL-051	Hard-coded tick spacing for the MAMO to cbBTC swap in DropAutomationV2	Informational	0.1	ACKNOWLEDGED
HAL-052	Discrepancy between documentation and implementation	Informational	—	SOLVED 10/22/2026
HAL-053	Floored withdraw leg minimums can undershoot the requested amount	Informational	—	SOLVED 10/15/2026

Findings & Tech Details

HAL-001 SOLVED

Owner cash-exit floor uses only the in-transaction TWAP, with no caller minimum or deadline

Low · 3.4 AO:A/AC:M/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N

DESCRIPTION

`StockAccountStrategy::withdraw` and `StockAccountStrategy::withdrawAll` protect each sell leg only with a minimum derived from the 180-second pool TWAP (or a Chainlink feed for cbBTC) read inside the transaction. Neither function accepts a caller-supplied absolute output bound or deadline, and the router call in `StockAccountStrategy::_sell` passes `block.timestamp` as the deadline. Two paths reach the owner's funds: a same-block sandwich can push execution to the floor, so the owner loses about the chosen tolerance plus any spot-over-reference gap; and a deflation held for one TWAP window lowers the reference and the execution price together, so the loss is not bounded by `maxWithdrawSlippageBps`. Both conditions are feasible outside market hours when pool counter-flow is limited.

SOL 882–897

```sol
882  function _sell(address token, uint256 amountIn, uint256 minOut) internal returns (uint256
     amountOut) {
883      ISwapRouter router = stockRegistry.aerodromeRouter();
884      IERC20(token).forceApprove(address(router), amountIn);
885      amountOut = router.exactInputSingle(
886          ISwapRouter.ExactInputSingleParams({
887              tokenIn: token,
888              tokenOut: address(asset),
889              tickSpacing: IPool(stockRegistry.tokenConfig(token).pool).tickSpacing(),
                 recipient: address(this),
890              deadline: block.timestamp,
891              amountIn: amountIn,
892              amountOutMinimum: minOut,
893              sqrtPriceLimitX96: 0
894          })
895      );
896  }
897
```

When an owner calls `StockAccountStrategy::withdrawAll`, `StockAccountStrategy::_planSells` computes each leg minimum as `legValue × (10,000 − maxSlippageBps) / 10,000`, where `legValue` comes from `StockAccountPriceChecker::getExpectedOut` reading the 180-second TWAP, and `StockAccountStrategy::_sell` passes `block.timestamp` as the router deadline. An attacker who holds the pool tick below fair value for one full TWAP window deflates both the reference and the per-leg floor together, so the owner's loss is not bounded by `maxWithdrawSlippageBps`; at live pool depth

the break-even notional is estimated at 11,000 to 12,000 USDC of NVDAc sold out of range and about 78,000 USDC in range, based on uniform-liquidity models and the project's pool documents. The attacker needs token inventory and cannot force or anticipate an exit because Base has no public mempool; a same-block sandwich captures approximately the tolerance plus the spot-over-reference gap. In-kind exits are not affected.

RECOMMENDATION

Let the owner bind a cash exit to an independent quote. Add a `minUsdcOut` parameter and a deadline to `StockAccountStrategy::withdrawAll`. Add a deadline to `StockAccountStrategy::withdraw` and a caller bound on what is sold, for example per-token maximum amounts from `previewWithdraw` checked against the planned amounts, because `withdraw` always pays `usdcAmount` and a deflated reference appears as extra tokens sold. Forward the caller-supplied deadline to the router. Derive front-end bounds from an independent off-chain or spot quote; `previewWithdraw` reads the same TWAP and gives no independent bound. Optionally reject a cash exit when the pool spot tick and the TWAP tick differ by more than a configured limit. Keep the per-leg floor as a second check. These changes affect the `IStockAccountStrategy` interface.

REMEDIATION COMMENT

Addressed in: 4492632a73f3f0b76d781d2ab569b3334d860b5d

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by adding a caller `minUsdcOut` and `deadline` to `withdrawAll`, a `deadline` plus sell caps for each token (`tokens`, `maxAmounts`, checked in `_checkSellCaps`) to `withdraw`, and forwarding the caller `deadline` to the router in `_sell`.

HAL-002 SOLVED

A dust balance of a listed token makes withdraw and withdrawAll revert

Low · 2.5 AO:A/AC:L/AX:L/R:P/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

`StockAccountStrategy::_sellable` includes every non-zero, non-Halted, priced balance in the sell plan with no minimum-value floor. `StockAccountStrategy::_executeSells` sells each leg with `amountOutMinimum = legValue × (10000 − s) / 10000` and no per-leg error handling. When a dust balance produces a swap that the pool rounds below the minimum output, the router reverts and the entire withdrawal reverts.

The same failure occurs without a donation when `withdraw` has a small shortfall, for example a withdrawal of the idle balance after the fee accrues, because `_planSells` then plans pro-rata legs of a few raw units that cannot clear their minimum.

SOL 768–779

```
768 for (uint256 i = 0; i < listed.length; i++) {
769     uint256 balance = IERC20(listed[i]).balanceOf(address(this));
770     if (balance == 0) continue;
771     if (stockRegistry.tokenConfig(listed[i]).status ==
    IStockAccountRegistry.TokenStatus.Halted) continue;
772
773     (bool priced, uint256 value) = _tryValue(checker, listed[i], balance);
774     if (!priced) continue;
775
776     tokens[count] = listed[i];
777     balances[count] = balance;
778     values[count++] = value;
779 }
```

Any address transfers 5 wei of NVDAc to the target account; at live prices that produces a leg value of about 11 raw USDC and a minimum of 10 at 500 bps. The Aerodrome exact-input router keeps at least one wei as the pool fee and rounds the output down, returning about 9 raw USDC, which is below the 10-unit minimum; the router reverts and the whole `withdrawAll` reverts. For cbBTC, legs up to about 18 to 19 wei fail at the same tolerance. `withdraw` includes a dust leg whenever the pro-rata share rounds to at least 1, a condition the donor can size for the expected call, and a withdrawal of the idle balance after fee accrual can produce the same small legs without a third-party transfer. The donor can repeat the transfer for the cost of gas. Any address can block the cash exit of a targeted account; `withdrawToken` and `withdrawAllInKind` are not affected, so funds remain reachable in kind.

RECOMMENDATION

Give every sell leg a minimum size. In `_sellable`, skip a balance whose reference value is below a registry `minSellValue` (for example 1 USDC) and leave it in the account for an in-kind exit. In `_planSells`, plan each leg at zero or at least `minSellValue` of reference value: move the share of a smaller leg to the largest leg, and raise the largest leg to `minSellValue` when the shortfall is smaller. The extra proceeds stay in the account as idle asset. Keep the per-leg minimum and the final `InsufficientProceeds` check.

REMEDIATION COMMENT

Addressed in: 73631187bb044169bf07c7d862a853a095219226

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by skipping balances worth less than `MIN_SELL_VALUE` (1 USDC) in `_sellable` and, in `_planSells`, folding legs below `MIN_SELL_VALUE` into the largest leg and raising the largest leg to at least `MIN_SELL_VALUE`, capped at its balance.

HAL-003 RISK ACCEPTED

Permissionless payFees in the sell token makes full-balance orders unfillable

Low · 2.5 AO:A/AC:L/AX:L/R:P/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

Any address can call `StockAccountStrategy::payFees` with no restriction, and `_isFeeToken` accepts any listed token that is not Halted as the fee denomination. When an open co-signed order sells the account's entire balance of a token, a `payFees` call in that token, or a `payFeesFromAny` call when the asset balance cannot cover the fee, reduces the balance below `sellAmount`, causing `StockAccountStrategy::isValidSignature` to revert with `SellExceedsBalance` at settlement. Each payment takes the whole fee backlog since the last payment.

SOL 256-258

```sol
256 function payFees(address token) external override {
257     _payFees(token);
258 }
```

Before an order settles, any address can call `payFees(sellToken)`, or `payFeesFromAny` when the asset balance cannot cover the fee. `_payFees` transfers the whole fee backlog since the last payment, converted into the sell token, from the sell-token balance to the fee recipient. When a co-signed order sells the full balance of a token (for example, a position-closing order with target 0 or a `SellOnly` wind-down), `isValidSignature` checks whether `order.sellAmount` exceeds `IERC20(sellToken).balanceOf(address(this))` after the fee has been taken; the balance is now below `sellAmount`, so the function reverts with `SellExceedsBalance`. The caller can repeat this for every new order the backend signs for the cost of gas. Orders that close a position (target 0, `SellOnly` wind-down, or basket removal) are the natural case, and an order that sells the whole asset balance is affected in the same way. No funds are lost, because the fee goes to the fee recipient. The backend can avoid the effect by sizing orders with a margin; that sizing policy is not encoded in the repository. Orders that leave at least the fee backlog in the sell token are not affected.

RECOMMENDATION

Size every co-signed sell order that sells all or nearly all of a token balance, the asset included, to at most the balance less a fee margin in that token. Compute the margin as `feeDueIn(sellToken)` at signing plus the fee that accrues in that token until the order's `validTo`, with a buffer for price movement during the order lifetime, because `_payFees` takes the whole backlog since `lastFeePaid` and any caller can trigger it through `payFees` or `payFeesFromAny`. Put this rule in the order service and add a test that pays the fee in the sell token between signing and settlement.

REMEDIATION COMMENT

RISK ACCEPTED: The `Moonwell` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> Operational control in the backend order service and spec v0.12: an order that sells an account's whole balance of a token is sized to balance minus `feeDueIn(sellToken)` minus the fee that accrues until `validTo`, so a permissionless `payFees` in the sell token cannot make it unfillable. The on-chain alternative (restricting the fee token) conflicts with the fee design accepted under the finding above; a griefer gains nothing and the fee reaches Mamo either way.

HAL-004 SOLVED

Upgrade does not migrate the stored principal

Low · 2.5 AO:A/AC:L/AX:M/R:P/S:U/C:N/A:N/I:H/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategy::initialize` and `MamoStrategyRegistry::upgradeStrategy`, where the new implementation introduces `principal` as a new storage slot that backs the deposit cap, but the initializer never writes it and the registry calls `upgradeToAndCall` with empty data. Proxy accounts created from the previously deployed implementation read `principal = 0` after the owner upgrades, so `_checkAccountValue` counts only deposits made after the upgrade against the cap, and the capital held before the upgrade is ignored.

SOL 605-610

```
605 function _checkAccountValue() internal view {
606     uint256 nav = getNAV();
607
608     if (nav < stockRegistry.minStrategyDeposit()) revert AccountBelowMinimum(nav);
609     if (principal > stockRegistry.maxStrategyDeposit()) revert
    DepositCapExceeded(principal);
610 }
```

The deployed implementation has no `principal` variable; it caps the account value (NAV) at `maxStrategyDeposit`. The owner creates an account and deposits 25,000 USDC, and no `principal` value is recorded. When the owner upgrades through `MamoStrategyRegistry::upgradeStrategy`, the registry calls `upgradeToAndCall` with empty data, no reinitializer runs, and `principal` reads 0 because the slot was absent in the prior storage layout. Any subsequent caller can then invoke `deposit`, and `_checkAccountValue` counts only deposits made after the upgrade against the cap, so the capital held before the upgrade is ignored. An account that already holds 25,000 USDC accepts a further 25,000 USDC, about doubling the per-account exposure limit documented in `docs/STOCK_ACCOUNT_PRICE_CHECKER.md`. No direct fund loss is shown; the number and value of live accounts are not known.

RECOMMENDATION

Seed `principal` once for each upgraded proxy before any funds move. Add a storage flag that `initialize` sets for new accounts. At the start of `deposit`, `depositToken`, `withdraw`, `withdrawToken` and `recoverERC20`, before any fee payment or token transfer, set `principal` to the current NAV and set the flag when the flag is not yet set. The deployed implementation capped NAV, so this keeps the earlier bound for the upgraded accounts. Add an upgrade test that starts from the storage layout of the deployed implementation and deposits after the upgrade.

REMEDIATION COMMENT

Addressed in: a3db6d56e3a4faf62efe51a6679946300d4efc96

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by redeploying the stock accounts system under strategy type id `7` through proposal `019_RedeployStockAccountSystem` instead of upgrading the `v1` (type id `5`) accounts, so no proxy ever moves from the `v1` layout to the implementation that introduces `principal`.

HAL-005 RISK ACCEPTED

Manipulable stock reward swap floor due to same pool TWAP reference

Low · 2.2 AO:A/AC:M/AX:M/R:N/S:U/C:N/A:N/I:N/D:N/Y:M

DESCRIPTION

The `MamoStakingStrategyV2._swapHop()` function executes one hop of a reward token route to MAMO during `MamoStakingStrategyV2.compound()`. It derives the hop's `amountOutMinimum` from `hop.checker.getExpectedOut()` reduced by the account slippage. The NatSpec of the global `MamoStakingRegistryV2.slippagePriceChecker` states that the reference used by `compound()` is anchored to an external oracle, because a quote read from the pool itself is self-referential when the quote and the swap use the same pool in the same transaction.

However, the `017_DeployStockDrop` proposal configures every stock reward route through `_stockRoute()` with a first hop that names its own checker, `STOCK_ACCOUNT_PRICE_CHECKER`, and swaps the stock into USDC on the stock pool. For stock tokens, `StockAccountPriceChecker.getExpectedOut()` returns a price derived from a 180 second `observe()` TWAP of the pool recorded for the token, which is the same stock pool that the hop swaps through. A price move within a single transaction does not change this reference, but no spot versus TWAP deviation check applies on this path, so a price held for the whole window becomes the reference. The minimum output of the following USDC to MAMO hop is chained from the expected output of the first hop, so it inherits the lowered value.

As a result, any actor able to hold a stock pool price down for at least 180 seconds lowers the minimum output of the whole stock route in proportion. If `compound()` is called for a staking account while the price is still held down, that account's stock rewards are sold at the depressed price without being held, and the account slippage is applied on top of the already lowered reference. If the AAPLc price is held 10% lower on the live pool, the account receives about 10% less MAMO than at the fair price. The loss is limited to the value of the stock rewards compounded while the price is held down.

RECOMMENDATION

Consider holding the stock reward in `MamoStakingStrategyV2._swapHop()` when a hop priced by `STOCK_ACCOUNT_PRICE_CHECKER` runs on a pool whose spot price deviates from a longer TWAP of the same pool by more than a configured threshold. Additionally, consider applying to stock compounds a backend price sanity check that compares the pool TWAP with the Chainlink equity feed during market hours before calling `MamoStakingStrategyV2.compound()`.

REMEDIATION COMMENT

RISK ACCEPTED: The `Moonwell` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> Same economics as the stock accounts' own exit floor (H35): a stock pool price held down for the full 180-second window lowers the reference with it. Exposure is limited to the stock rewards compounded during such a window. The on-chain lever is the guardian halt in `StockAccountRegistry`, which now makes `compound()` hold the reward instead of selling it (H52 fix). The backend is required to apply the same price-sanity rule the stock accounts use (pool TWAP against the Chainlink equity reference, or against the last print when the market is closed) before calling `compound()`, and to skip the stock while it fails. An on-chain spot-vs-TWAP gate on the stock hop remains an option for a later checker revision.

HAL-006 SOLVED

Registry administrator can replace the price checker or router and loosen limits for every account at once

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:C/Y:N

DESCRIPTION

The issue originates in `StockAccountRegistry::setPriceChecker` and `StockAccountRegistry::setAerodromeRouter`. Both functions accept any contract with code and take effect in the same transaction with no delay, pending step or owner opt-out. Every account reads its price checker, router, deviation band and slippage cap live from the registry, so one administrator transaction reaches every account at once. The role holder is a 2-of-3 Safe with no module, guard or timelock, and the same Safe holds `GUARDIAN_ROLE`, so the pause is not an independent brake.

SOL 109-126

```
109 function setAerodromeRouter(ISwapRouter newRouter) external onlyRole(DEFAULT_ADMIN_ROLE)
    whenNotPaused {
110     if (address(newRouter) == address(aerodromeRouter)) revert AlreadySet();
111     _setAerodromeRouter(newRouter);
112 }
113
114 /// @notice Sets the price checker used to validate swaps
115 function setPriceChecker(ISlippagePriceChecker newPriceChecker)
116     external
117     onlyRole(DEFAULT_ADMIN_ROLE)
118     whenNotPaused
119 {
120     if (address(newPriceChecker) == address(priceChecker)) revert AlreadySet();
121     _setPriceChecker(newPriceChecker);
122
123     for (uint256 i = 0; i < _tokens.length; i++) {
124         if (_tokenConfig[_tokens[i]].status != TokenStatus.Halted)
    _requirePriceable(_tokens[i]);
125     }
126 }
```

Two Safe signers act together, first granting `BACKEND_ROLE` to a controlled address on `MamoStrategyRegistry` and directing fees to it through `setFeeRecipient`, then calling `setPriceChecker` with a checker that returns very large quotes. The permissionless `payFees` and `payFeesFromAny` then send `min(converted due, balance)` of USDC and each priced listed token to the controlled recipient, continuing through every token because credited time rounds to near zero. A separate transaction calls `setAerodromeRouter` with a malicious router; any subsequent `withdrawAll` from any account transfers the sell proceeds to the attacker, because `withdrawAll` checks only that the final asset balance is non-zero. The administrator can also disable the deviation band by setting `maxDeviationBps` to 10,000 bps. The 2-of-3 Safe has no timelock and the same Safe controls the guardian pause, so neither mechanism provides a delay. With a replaced price checker the

permissionless fee path can drain USDC and all priced listed token balances of every account in one transaction.

RECOMMENDATION

Add a fixed delay to the registry setters that loosen account protections. Make `setPriceChecker`, `setAerodromeRouter`, `setMaxDeviationBps` and `setMaxBackendSlippageBps` record a pending value and emit it, and apply the value only after a minimum delay (for example 48 hours) in a `whenNotPaused` step, so that owners can exit first. Give `GUARDIAN_ROLE` to keys separate from the administrator, so that the guardian can pause a pending change. In `_sell`, measure the asset balance before and after the swap and require the increase to be at least `minOut`.

REMEDIATION COMMENT

Addressed in: 7192df285e75f192c140a282089bd72c0933ec3c

SOLVED: The router payment check was verified in `StockAccountStrategy.sol:979–1003`. The reported immediate registry admin changes remain in `StockAccountRegistry.sol:124–165`. The submitted source does not establish an enforced delay or an independent guardian. The following statement was provided by the `Moonwell` team:

> The withdrawal sell path measures the asset balance the router actually moved and reverts RouterUnderpaid when it is below the floor, instead of trusting the router's return value; the registry also cross-checks a new checker's quoteAsset (2c9a6a84f32a85f9a2115084d812befc656657cc). The instant-swap admin powers remain by design; the operational control is a delay module on the admin Safe with the guardian on a separate Safe.

HAL-007 RISK ACCEPTED

Centralization risks

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:C

DESCRIPTION

The stock drop contracts implement several privileged functions that give a small set of keys immediate, unconstrained control over the rewards owed to stakers. The F-MAMO Safe owns `DropAutomationV2` and `MultiRewards` and holds both `DEFAULT_ADMIN_ROLE` and `GUARDIAN_ROLE` on `MamoStakingRegistryV2`. `BACKEND_ROLE` on that registry is held by `STRATEGY_MULTICALL`, and because `DEFAULT_ADMIN_ROLE` is the admin of every other role, the Safe can grant itself `BACKEND_ROLE` or revoke any other guardian at will. For example:

- The `DropAutomationV2.recoverERC20()` function allows the owner to transfer any token held by the contract, in any amount, to any address, including the protocol fees that accumulate between drops, the remainders kept for the next drop and balances whose notification failed. The `DropAutomationV2.withdrawGauge()` function allows the owner to send the protocol's staked LP tokens to any recipient, and `DropAutomationV2.setDedicatedMsgSender()` allows the owner to replace the address that runs drops and supplies their minimum swap outputs.
- The `MultiRewards.recoverERC20()` function allows the same owner to withdraw any reward token other than MAMO, including cbBTC and the WETH, USDC and stock tokens added by the stock drop proposals, while a reward period is active. Once a token is drained, `MultiRewards.getReward()` and `MultiRewards.exit()` revert for every staker owed more of that token than the contract holds, so none of their reward tokens can be claimed until the balance is restored. This also blocks `MamoStakingStrategyV2.withdrawAll()`, `MamoStakingStrategyV2.withdrawRewards()`, `MamoStakingStrategyV2.compound()` and `MamoStakingStrategyV2.reinvest()`.
- The `MamoStakingRegistryV2.setDEXRouter()`, `MamoStakingRegistryV2.setSlippagePriceChecker()` and `MamoStakingRegistryV2.setRoute()` functions allow `DEFAULT_ADMIN_ROLE` to replace the router, the price checker and any per-hop router or checker that every staking account reads during `MamoStakingStrategyV2.compound()`, with no allow-list and no delay.
- The `MamoStakingStrategyV2.setStakingRegistry()` function allows the current registry's `DEFAULT_ADMIN_ROLE` to move every staking account to a new registry in one batch.
- The `MamoStakingRegistryV2.setDefaultSlippage()` and `MamoStakingRegistryV2.updateRewardTokenPool()` functions allow `BACKEND_ROLE` to widen the minimum-output tolerance of every account that keeps the default up to 2500 basis points and to choose the pool, and therefore the tick spacing, a reward token is swapped through.

- The `MamoStakingStrategyV2.reinvest()` function, callable by `BACKEND_ROLE` on `MamoStakingRegistryV2`, approves each reward token other than MAMO to a destination chosen by the caller and then calls its deposit function. A destination is accepted when its own `owner()` answer matches the account owner and `MamoStrategyRegistry.isUserStrategy()` returns true, and `MamoStrategyRegistry.addStrategy()` registers any contract whose own `getImplementation()`, `owner()`, `mamoStrategyRegistry()` and `strategyTypeId()` answers pass its checks, without verifying its code. `STRATEGY_MULTICALL` holds `BACKEND_ROLE` on both registries and forwards arbitrary calls for its owner, the `MAMO_COMPOUNDER` EOA, so that single key can register a contract it controls for any user and have `reinvest()` hand it that user's claimed cbBTC, WETH, USDC and stock token rewards. No role on `MamoStrategyRegistry` can remove such an entry.

While these functions are needed for maintenance and incident response, they take effect in the same transaction, the Safe is a 2-of-3 multisig with no timelock, and the registry pause is held by the same Safe, so it is not an independent brake. A compromised or malicious key set could withdraw the reward balances held for distribution in `DropAutomationV2` and `MultiRewards`, route every compounding swap through a router or price checker that returns little or nothing, and stop reward claims for every affected staker until the balances are refunded. Staked MAMO cannot be transferred out through these paths, because `MultiRewards.recoverERC20()` rejects the staking token, `MamoStakingStrategyV2.compound()` reverts when a registry lists MAMO as a reward token, and `MamoStakingStrategyV2.withdraw()` keeps working after a reward balance is drained. However, the same owner can still make `MultiRewards.withdraw()` revert for earlier stakers by re-adding a removed reward token through `MultiRewards.addReward()`.

RECOMMENDATION

Several remedial strategies can be employed, including but not limited to:

- Place the F-MAMO Safe behind a timelock or delay module for its administrative calls on `DropAutomationV2`, `MultiRewards` and `MamoStakingRegistryV2`, so that stakers and account owners can react before a change takes effect.
- Assign `GUARDIAN_ROLE` on `MamoStakingRegistryV2` to keys independent of `DEFAULT_ADMIN_ROLE`, keep the pause free of any delay, and monitor the events these functions emit.
- In `MamoStakingStrategyV2.reinvest()`, send each reward token only to a destination the account owner has approved, and hold the reward otherwise.
- Keep `DropAutomationV2.recoverERC20()` and `MultiRewards.recoverERC20()` available for held balances, and document the trust placed in each role for stakers.

REMEDIATION COMMENT

RISK ACCEPTED: The `Moonwell` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> Design: the F-MAMO Safe owns `DropAutomationV2` and `MultiRewards` and administers `MamoStakingRegistryV2` so that incident response (repointing a router or checker, recovering a stuck balance, pausing) is immediate. The accepted mitigations are operational: the admin role moves behind a delay module on the Safe, the guardian role to a separate key set, and the trust assumptions are documented. Staked MAMO cannot leave through any of the listed paths, as the finding notes.

HAL-008 SOLVED

No sequencer uptime check on the TWAP and cbBTC Chainlink price paths

Informational · 1.9 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:L/D:M/Y:N

DESCRIPTION

The issue originates in `StockAccountPriceChecker::_twapTick`, where `pool.observe()` is called without any sequencer uptime gate or post-restart grace period. After a Base sequencer outage, the mean tick returned by `_twapTick` equals the pre-outage value until post-restart swaps fill the configured window. For the cbBTC Chainlink path, the live `SlippagePriceChecker` contains `_requireSequencerUp`, but the guard is a no-op because `sequencerUptimeFeed` is not set on the live deployment; proposal 016 configures the cbBTC pair without arming the guard or asserting its presence.

SOL 203-215

```
203     function _twapTick(ICLPool pool, uint32 window) internal view returns (int24 tick) {
            uint32[] memory secondsAgos = new uint32[](2);
204         secondsAgos[0] = window;
205         secondsAgos[1] = 0;
206         try pool.observe(secondsAgos) returns (int56[] memory cum, uint160[] memory) {
207             int56 delta = cum[1] - cum[0];
                int56 w = int56(uint56(window));
208             tick = int24(delta / w);
209             if (delta < 0 && delta % w != 0) tick--;
210         } catch {
211             revert InsufficientObservations(address(pool), window);
212         }
213     }
214
```

During and right after a Base sequencer outage, stock accounts accept prices that do not reflect the market. On the TWAP path the mean tick at restart equals the pre-outage tick until post-restart trades fill the 180-second window. On the cbBTC path a pre-outage BTC/USD round inside the 3,600-second heartbeat is accepted. A co-signed order with at least 5 minutes of validity remaining can settle against a stale reference after an adverse move, a loss bounded by the order size and the deviation band; owner exits right after the restart use a stale floor, or a manipulated one if a manipulation was in place when the outage began. NAV, the fee base and `depositToken` principal for cbBTC use the stale round. When the price of a held token is refused, `withdrawAll` pays out the asset and the priced legs and leaves the affected holdings in the account; `withdraw` reverts only when the remaining priced legs cannot cover the requested amount; in-kind exits stay available. A sequencer outage is rare and not attacker-controlled. On the deployed system the cbBTC gap is deployment state: the repository checker contains a guard that is disabled until proposal 013 sets it. Cash exits right after the restart use a stale floor.

RECOMMENDATION

Fix the two paths separately. `cbBTC` (Chainlink) path: execute proposal 013 (checker upgrade and `setSequencerUptimeFeed` with the Base uptime feed and a grace period of at least the 30-minute `MAX_ORDER_VALIDITY`) before or with 016, and add a 016 `validate()` assertion that the checker's `sequencerUptimeFeed` is the Base uptime feed and `sequencerGracePeriod` is at least `MAX_ORDER_VALIDITY`. TWAP path: give `StockAccountPriceChecker` the Base sequencer uptime feed and refuse `PoolTwap` quotes while the sequencer is down and for a grace period of at least `max(twapWindow, MAX_ORDER_VALIDITY)` after it returns. Make sure that `payFees` and `payFeesFromAny` leave the fee clock unchanged while a held token is unpriced, so that the account value reading zero during the grace period cannot reset it. Document the grace-period behaviour: orders that price an affected token revert, valuation drops the affected holdings, `withdraw` sells only the priced legs and reverts when they cannot cover the amount, `withdrawAll` pays out the asset and the priced legs and leaves the affected holdings in the account, and the in-kind exits stay available.

REMEDIATION COMMENT

Addressed in: `1e15f147d5fd83f2efc039fc01da47b4e5162b94`

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by gating every `PoolTwap` quote in `StockAccountPriceChecker` on an immutable `sequencerUptimeFeed` and `sequencerGracePeriod` through `_requireSequencerUp`.

HAL-009 SOLVED

Any member of the shared backend role can set the fee recipient of every account

Informational · 1.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:H

DESCRIPTION

The issue originates in `StockAccountStrategy::setFeeRecipient`. The `onlyBackend` modifier calls `BaseStrategy::_isBackend`, which is a plain `hasRole(BACKEND_ROLE)` check on `MamoStrategyRegistry`. That role is shared across all Mamo products and has seven members on Base, including `STRATEGY_MULTICALL`, whose owner, the MAMO_COMPOUNDER EOA, can make arbitrary calls through it.

SOL 229-235

```
229 function setFeeRecipient(address newRecipient) external override onlyBackend {
230     if (newRecipient == address(0)) revert ZeroAddress();
231
232     emit FeeRecipientUpdated(feeRecipient, newRecipient);
233
234     feeRecipient = newRecipient;
235 }
```

If any member of the shared `BACKEND_ROLE` is compromised, the attacker can call `StockAccountStrategy::setFeeRecipient` once per account to redirect management fees across all stock accounts; the role includes `STRATEGY_MULTICALL`, an arbitrary-call contract whose owner EOA sits outside the stock account product. Setting a fee recipient that USDC refuses to credit also prevents `deposit`, `depositToken`, `withdraw` and `withdrawAll` from completing for every account using that recipient. At 100 bps per year on the per-account cap, the diverted amount is the accrued fee portion until the next authorized rotation; principal is not accessible through this function.

RECOMMENDATION

Scope the fee recipient to the stock product. Replace `onlyBackend` on `StockAccountStrategy::setFeeRecipient` with a role held on `StockAccountRegistry`, for example a `FEE_MANAGER_ROLE` or `DEFAULT_ADMIN_ROLE` behind a timelock. Alternatively, store one fee recipient in `StockAccountRegistry` under that controlled setter and let accounts read it live, so that no member of the shared `BACKEND_ROLE` can change it per account.

REMEDIATION COMMENT

Addressed in: b2d9bcc0e4cd6b5271d0362edd47fd3115e80455

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by removing `StockAccountStrategy::setFeeRecipient` and the per-account `feeRecipient`, and having every account pay `stockRegistry.feeRecipient()`, set only through `StockAccountRegistry::setFeeRecipient` restricted to `DEFAULT_ADMIN_ROLE`.

HAL-010 ACKNOWLEDGED

payFees caller chooses the fee token at a manipulable TWAP

Informational · 1.1 AO:A/AC:H/AX:M/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategy::payFees`, which carries no access control and accepts any non-halted token as the fee denomination. The fee amount conversion in `_feeAmount` calls `getExpectedOut(due, address(asset), token)`, which uses the 180-second pool TWAP of the requested token against the asset with no deviation bound, and `_payFees` caps the transfer only at the account's balance of that token. Any caller can trigger fee payment at any time in the asset or any listed token that is not Halted, directly with `payFees(token)`, and through `payFeesFromAny` or a deposit when the asset balance cannot cover the fee.

SOL 256-258

```
256 function payFees(address token) external override {
257     _payFees(token);
258 }
```

A caller who first depresses the TWAP of a chosen token causes `_feeAmount` to compute a larger token amount for the same USDC-denominated fee due, and `_payFees` transfers up to the full balance to the fee recipient; for example, with 5,000 USDC of a stock token, about 16 USDC due, and the token quoted at 1/1000 of fair value, the whole token balance is taken for roughly 31% of the period. One manipulation window can be used against every account that holds the token, and a small distortion causes an overcharge equal to the fee times the distortion. Moving the TWAP of a live equity pool (holding 0.8 to 1.45 million USDC) by an order of magnitude and holding it against arbitrage for the 180-second window is the precondition for a significant extraction; a small distortion causes only a proportional overcharge. The value goes to the fee recipient, not the caller, so an unrelated caller only pays the manipulation cost; the case matters mainly for an actor aligned with the fee recipient, which a backend role member can set.

RECOMMENDATION

Remove third-party control over fee payments in tokens other than the asset.

- In `_payFees`, take a token other than the asset only when `msg.sender` is the account itself, the owner or a backend role member. Any caller can still settle in the asset. This one check covers `payFees`, the listed-token walk in `payFeesFromAny` and the settlement before a deposit.

- In `deposit`, when another caller deposits and the asset balance does not cover the backlog, pay the remainder out of the deposited asset, computed on the account value before the transfer, so that new funds do not pay for the earlier period.
- In `appDataDocument` and the matching appData check in `isValidSignature`, make the order post-hook pay the fee in the asset. The post-hook reaches the account through the CoW `HooksTrampoline`, which also runs hooks from orders that other parties create.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> Any caller may choose the fee token at the reference price by design: the fee is valued on the whole account and the post-hook pays it in the token the order buys, which is what feeds the weekly stock drop; forcing USDC would defeat that design. The exposure is bounded to the fee amount at a TWAP that the per-order and withdrawal paths already accept as the reference. Documented in the contract spec and runbook.

HAL-011 SOLVED

Halted stock rewards still sold due to missing status check in compound

Informational · 1.1 AO:A/AC:M/AX:H/R:N/S:U/C:N/A:N/I:N/D:N/Y:M

DESCRIPTION

The `MamoStakingStrategyV2.compound()` function swaps every reward token listed in `MamoStakingRegistryV2` into MAMO through `MamoStakingStrategyV2.compoundRewardToken()`, and `MamoStakingStrategyV2._swapHop()` derives each hop's minimum output from `hop.checker.getExpectedOut()` minus the account slippage. The `017_DeployStockDrop` proposal configures, through `_stockRoute()`, a first hop for every B20 stock reward token that swaps the stock into USDC on the stock pool, priced by `STOCK_ACCOUNT_PRICE_CHECKER` from a 180 second TWAP of that same pool.

In the stock account system, `StockAccountRegistry.setTokenStatus()` lets the guardian halt a token, which its NatSpec describes as a remediation lever that stays available while the registry is paused. `StockAccountStrategy` honors that status by excluding Halted tokens from valuation, sell plans and fee payment, while `StockAccountPriceChecker._toQuote()` keeps pricing Halted tokens and leaves the enforcement to each caller.

However, neither `MamoStakingStrategyV2` nor `MamoStakingRegistryV2` reads the token status in `StockAccountRegistry`, and the `onlyBackend` modifier only checks `MamoStakingRegistryV2.paused()`. A halt of a stock token, or a pause of `StockAccountRegistry`, therefore does not stop `MamoStakingStrategyV2.compound()` from selling that token: the checker returns the TWAP of the halted pool and the swap executes on that same pool.

As a result, when the guardian halts a stock because its pool price is no longer reliable, for example while any actor holds a thin pool price down for the TWAP window, the routine backend `compound()` call keeps selling every staking account's rewards in that stock at the flagged price. The loss is limited to the stock rewards processed while the halt is in force.

RECOMMENDATION

Consider holding a stock reward token in `MamoStakingStrategyV2.compoundRewardToken()` when its `StockAccountRegistry` status is Halted or the registry is paused, so that the existing try/catch keeps the token in the account. Alternatively, because `StockAccountRegistry.setTokenStatus()` and `MamoStakingRegistryV2.removeRewardToken()` are restricted to different accounts, consider an operational procedure that removes the token from `MamoStakingRegistryV2` right after every halt.

REMEDIATION COMMENT

Addressed in: `12c31c4bc99d2df02850b82793d46cbc622eccff`

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by adding `_checkStockHop()` to `MamoStakingStrategyV2._swapHop()`, which reads `StockAccountRegistry.tokenConfig()` for any hop priced by the stock checker and reverts with "Stock not sellable" unless the token is `Active` or `SellOnly`.

HAL-012 SOLVED

Account value that excludes halted or unpriceable holdings feeds the range check and the fee, and a zero value resets the fee clock

Informational · 1.0 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:L/I:N/D:N/Y:L

DESCRIPTION

`StockAccountStrategy::_valuation` skips any token whose status is `Halted` and any token whose price quote reverts, returning only the value of the remaining holdings without a completeness signal. That partial value is passed to `_checkRange` as the denominator for weight checks and to `_payFees` and `_payFeesFromAny` as the fee base. When no holding is priced, both fee functions advance `lastFeePaid` to the current time, erasing the unsettled period.

SOL 343-366

```
343 function _valuation() internal view returns (address[] memory tokens, uint256[] memory
    values, uint256 nav) {
344     tokens = stockRegistry.allTokens();
345     values = new uint256[](tokens.length);
346     nav = asset.balanceOf(address(this));
347
348     ISlippagePriceChecker checker = stockRegistry.priceChecker();
349
350     for (uint256 i = 0; i < tokens.length; i++) {
351         uint256 balance = IERC20(tokens[i]).balanceOf(address(this));
352         if (balance == 0) {
353             continue;
354         }
355
356         if (stockRegistry.tokenConfig(tokens[i]).status ==
    IStockAccountRegistry.TokenStatus.Halted) {
357             continue;
358         }
359
360         (bool priced, uint256 value) = _tryValue(checker, tokens[i], balance);
361         if (!priced) continue;
362
363         values[i] = value;
364         nav += value;
365     }
366 }
```

While a holding is excluded, any caller can invoke `payFees` or `payFeesFromAny` and settle the backlog at the reduced value; when nothing else is priced, the due is zero and `_payFees` or `_payFeesFromAny` sets `lastFeePaid` to now, erasing the entire accrued period, for example about 20.55 USDC per 30 days for a 25,000 USDC holding at 100 bps. The forgone fee stays with the account owner, and the window ends when the token is re-activated or priced again. `setTokenPool` requires the token to be `Halted`, so a planned pool change creates a predictable window. In `_checkRange`,

weights are measured against the reduced denominator: buy legs reach their ceilings at a lower absolute value and rebalancing stops in accounts where the halted share exceeds the band threshold (50% for a 1,000 bps leg at the live 1,000 bps band), while sell floors are correspondingly looser. Exits are not affected; the fail-open read keeps them working by design.

RECOMMENDATION

Make `_valuation` report whether it is complete: return a flag that is false when a non-zero balance was skipped because it is Halted or because its quote reverted. Use the flag where a partial value is unsafe.

- In `payFees` and `payFeesFromAny` called by any address other than the account itself, leave `lastFeePaid` unchanged while the value is incomplete. Reset the clock without a payment only when the account holds no listed balance at all, based on raw balances.
- In `_settleFeesBeforeDeposit` and the owner exits, keep settling the fee on the priced part and moving the clock to now, so that new funds never pay for the earlier period and an owner does not leave ahead of the fee. The fee on the excluded holding for that period is then not charged.
- In `_checkRange`, revert when a held non-halted token cannot be priced, and compare the weights of the other legs with targets renormalized over the non-halted part of the basket (scale each target by `10000 / (10000 – haltedTargetBps)`).

Settle fees on affected accounts before a planned halt.

REMEDIATION COMMENT

Addressed in: 392dac8da58500b121a9ae67d1a6377bdd862b51

SOLVED: The order and fee guards were verified in `StockAccountStrategy.sol:315–377` and `StockAccountStrategy.sol:638–761`. The reported handling of Halted holdings remains: they are excluded from valuation and fee timing, and range targets are not adjusted for them. The following statement was provided by the `Moonwell` team:

> The valuation reports completeness; isValidSignature refuses orders with ValuationIncomplete while any held non-Halted token cannot be priced; the fee is deferred rather than charged on a partial base while incomplete (f9c8b8988d254a99804a7af311a460c123e6d54d); the fee clock catches up only when the account holds no asset and no non-Halted token (2c5b257595a0db4294747a486df828d613216fe7); a balance that cannot be read counts as unpriced (985ee0ff8d21a67386ada2d21cf40d694b2a1df0).

HAL-013 ACKNOWLEDGED

Single-step ownership transfer process

Informational · 1.0 AO:S/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N

DESCRIPTION

The `MamoStakingStrategyV2` contract inherits `BaseStrategy`, which inherits OpenZeppelin's `OwnableUpgradeable` and restricts the account exits (`withdraw()`, `withdrawAll()`, `withdrawRewards()`, `recoverERC20()`) and `MamoStrategyRegistry.upgradeStrategy()` to the strategy owner. `BaseStrategy.transferOwnership()` first moves the registry record through `MamoStrategyRegistry.updateStrategyOwner()` and then transfers ownership in a single step:

SOL

```
function transferOwnership(address newOwner) public override onlyOwner {
    mamoStrategyRegistry.updateStrategyOwner(newOwner);

    super.transferOwnership(newOwner);
}
```

Because the transfer is single-step, the receiving address is never required to accept ownership, and `MamoStrategyRegistry.updateStrategyOwner()` applies no check to the new owner other than rejecting the zero address. If an owner passes a mistyped address, the strategy's own address or the registry address, the call succeeds and the previous owner loses access to every exit at once. No administrator or registry function can return ownership, so the staked MAMO of that account becomes unrecoverable. `BaseStrategy.renounceOwnership()` already reverts, but a transfer to an inaccessible address has the same effect.

Instances of this issue can be found in:

- `BaseStrategy.transferOwnership()`, inherited without override by `MamoStakingStrategyV2`

RECOMMENDATION

Consider making `BaseStrategy` inherit OpenZeppelin's `Ownable2StepUpgradeable` and removing the `mamoStrategyRegistry.updateStrategyOwner()` call from `BaseStrategy.transferOwnership()`, so that it only records the pending owner. Override `acceptOwnership()` to call `mamoStrategyRegistry.updateStrategyOwner(msg.sender)` before completing the transfer, so that the registry record moves only when the new owner accepts. `Ownable2StepUpgradeable` keeps its pending owner in namespaced storage, so the storage layout of existing proxies is unchanged. Keep `BaseStrategy.renounceOwnership()` disabled.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> `BaseStrategy.transferOwnership()` is the fleet-wide pattern shared by every Mamo strategy type, not specific to the stock drop. Moving `BaseStrategy` to a two-step transfer is planned for a future `BaseStrategy` revision across all strategy types, not for this release.

HAL-014 ACKNOWLEDGED

cbBTC valued with the BTC/USD feed

Informational · 0.9 AO:A/AC:L/AX:H/R:P/S:U/C:N/A:M/I:N/D:L/Y:N

DESCRIPTION

The issue originates in `DeployStockAccountSystem::_configureExistingChecker`, which registers cbBTC with a BTC/USD Chainlink feed as the first hop and a reversed USDC/USD feed as the second hop, with no cbBTC/BTC intermediate feed or depeg check. The deployment script describes this as the two-hop shape that other Mamo configurations use; it does not state that cbBTC is treated as equal to BTC, and no project document describes a depeg procedure.

During a cbBTC depeg, no price path reflects the deviation from BTC: NAV, the fee base, `depositToken` principal and order floors all use the BTC price for cbBTC.

SOL 330-358

```sol
/// @dev Two hops, feed -> USD -> USDC, the shape every other Mamo config uses. cbBTC is the only
///      Chainlink entry on the launch list; the loop is over the config so adding one needs no edit
function _configureExistingChecker() internal {
    StockAccountsConfig.DeploymentConfig memory cfg = deployConfig.getConfig();
    StockAccountsConfig.TokenListEntry[] memory entries = deployConfig.loadTokenList();

    ISlippagePriceChecker existing = ISlippagePriceChecker(addresses.getAddress(cfg.existingPriceChecker));
    address asset = addresses.getAddress(cfg.asset);
    address usdcUsdFeed = addresses.getAddress("CHAINLINK_USDC_USD");

    for (uint256 i = 0; i < entries.length; i++) {
        if (keccak256(bytes(entries[i].source)) != keccak256(bytes("Chainlink"))) continue;

        if (existing.tokenPairOracleInformation(entries[i].token, asset).length == 0) {
            ISlippagePriceChecker.TokenFeedConfiguration[] memory feeds =
                new ISlippagePriceChecker.TokenFeedConfiguration[](2);
            feeds[0] = ISlippagePriceChecker.TokenFeedConfiguration({
                chainlinkFeed: entries[i].chainlinkFeed,
                reverse: false,
                heartbeat: entries[i].heartbeat
            });
            feeds[1] = ISlippagePriceChecker.TokenFeedConfiguration({
                chainlinkFeed: usdcUsdFeed,
                reverse: true,
                heartbeat: USDC_USD_HEARTBEAT
            });

            existing.addTokenConfiguration(entries[i].token, asset, feeds);
        }

```

During a cbBTC depeg, the BTC/USD feed continues to supply the price for all stock-account cbBTC reads: NAV, the fee base and `depositToken` principal overstate cbBTC holdings, co-signed buy orders

pass `isValidSignature` at the BTC-derived floor so they can fill up to the depeg above the real market price, and cash exits that include a cbBTC sell leg revert because the floor exceeds the pool price. The configuration prices cbBTC as if it were equal to BTC, and no project document states this assumption. A depeg is an external and rare event; losses from buys are bounded by the depeg size and the order size. In-kind exits continue to work, and the guardian can halt cbBTC to remove it from valuation and enable cash exits.

RECOMMENDATION

Use a `cbBTC`-specific reference for `cbBTC` when Chainlink provides one on Base, either a `cbBTC`/USD feed or a `cbBTC`/BTC hop before BTC/USD. Until then, state the `cbBTC` = BTC assumption in the deployment documentation and add a monitored guardian procedure that sets `cbBTC` to `SellOnly` or Halted on a depeg.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> Moot since proposal 018 (2026-10-13) halted `cbBTC`, and `cbBTC` is not on the redeployed system's listing. If it is ever re-activated the assumption is stated in the runbook and the guardian halt is the depeg procedure.

HAL-015 SOLVED

Deposit cap limits tracked principal, not account value; full in-kind exit resets principal when a transfer is skipped

Informational · 0.8 AO:A/AC:L/AX:M/R:P/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategy::_checkAccountValue` and `StockAccountStrategy::withdrawAllInKind`. `_checkAccountValue` compares `principal` against `maxStrategyDeposit` by design, so that gains never block a top-up. Account value can therefore exceed the cap through price appreciation, direct ERC-20 transfers, or a full exit that leaves a balance in the account.

The price-checker documentation still states that the 25,000 USDC deposit cap bounds what one order or one exit can move, which does not hold for an account above the cap, because the exposure scales with the account value. Additionally, `withdrawAllInKind` sets `principal = 0` unconditionally before sending any balances. `_sendAllInKind` silently skips a failed transfer, for example when the issuer freezes the holder, so `principal` reads zero while the skipped balance stays in the account, and `principal` no longer matches capital deposited less capital withdrawn.

SOL 190-202

```sol
function withdrawAllInKind() external override onlyOwner {
    try this.payFeesFromAny() {} catch {}

    principal = 0;

    address to = owner();
    _sendAllInKind(address(asset), to);

    address[] memory tokens = stockRegistry.allTokens();
    for (uint256 i = 0; i < tokens.length; i++) {
        _sendAllInKind(tokens[i], to);
    }
}
```

For example, an account that receives 25,000 USDC by deposit and 75,000 USDC by direct transfer has a value of 100,000 USDC and a `principal` of 25,000 USDC. A co-signed order that sells 50,000 USDC passes `isValidSignature` because the sell-amount check uses the 100,000 USDC balance and the deviation band uses the 100,000 USDC NAV, not the deposit cap, so the order validates at twice the documented single-order exposure limit. Separately, when the owner calls `withdrawAllInKind` and a held stock token cannot be transferred because its issuer froze the holder, `_sendAllInKind` skips the transfer silently. `principal` reads zero although the skipped balance is still in the account, so a further deposit of up to the full 25,000 USDC cap is accepted on top of it. In

the in-kind exit, a skipped transfer leaves its balance in the account while `principal` reads zero, so `principal` no longer matches capital deposited less capital withdrawn. A plain token transfer already raises holdings without changing `principal`, so this is not a new way past the cap. No direct loss of funds occurs, and the funds concerned belong to the account owner.

RECOMMENDATION

Update `docs/STOCK_ACCOUNT_PRICE_CHECKER.md` so that the per-order and per-exit exposure analysis uses the account value, which can exceed `maxStrategyDeposit`, or add a registry per-order value limit in `isValidSignature` if the analysis must keep a fixed bound. At the end of `withdrawAllInKind` and `withdrawAll`, set `principal` to the lower of its current value and `getNAV()` after the transfers, so that an emptied account returns to zero and a balance that stays in the account keeps its value in `principal`. A balance that cannot be priced at the exit counts as zero, as it does in NAV.

REMEDIATION COMMENT

Addressed in: 05b3af25d4fd68dab0826001f15b093ac0f24f89

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by setting `principal = Math.min(principal, getNAV())` after the sweep in `withdrawAllInKind`, reducing `principal` by what `withdrawAll` ships, adding a registry `maxOrderValue` enforced in `_checkRange`, and correcting `docs/STOCK_ACCOUNT_PRICE_CHECKER.md`.

HAL-016 SOLVED

Reward dust left unstreamed due to rounding on notifications during an active period

Informational · 0.8 AO:A/AC:L/AX:M/R:P/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

The `DropAutomationV2._notify()` function funds each MultiRewards stream through `MultiRewards.notifyRewardAmount()`. Before notifying, it reads the stream duration and trims the amount to a multiple of it, keeping the remainder for the next drop, so that the notified amount streams in full.

However, the trim only makes the new amount a multiple of the duration. When a drop lands before the current period ends, `MultiRewards.notifyRewardAmount()` computes the new rate as `(reward + leftover) / rewardsDuration`, where `leftover` is the unstreamed part of the active period (`remaining * rewardRate`), which is not a multiple of the duration. The remainder `leftover % rewardsDuration` is truncated from the rate and never streams, although the tokens already sit in MultiRewards. `DropAutomationV2._notify()` reads only the duration from `MultiRewards.rewardData()`, although the same call also returns `periodFinish` and `rewardRate`.

Each notification that lands before `periodFinish` leaves up to `rewardsDuration - 1` base units unstreamed. With the 7-day duration of the deployed streams, that is up to 604,799 units, about 0.006 cbBTC for the 8-decimal cbBTC stream. For example, a notification made while 27,326,910 units are still owed from the active period leaves 110,910 units unstreamed. Fifty-six drops about one day apart, each funding about 0.07 cbBTC, leave 17,206,572 units (about 0.17 cbBTC) unstreamed. `createDrop()` has no minimum interval, so the frequency depends on the off-chain schedule: with drops every 7 days only a drop that lands shortly before `periodFinish` is affected, while any shorter cadence affects every drop. The owner can recover the unstreamed balance of reward tokens other than MAMO with `MultiRewards.recoverERC20()` and send it back to `DropAutomationV2` for the next drop. The MAMO remainder cannot be recovered because MAMO is the staking token.

RECOMMENDATION

In `DropAutomationV2._notify()`, also read `periodFinish` and `rewardRate` from `MultiRewards.rewardData()`. When `block.timestamp < periodFinish`, compute the leftover as `(periodFinish - block.timestamp) * rewardRate`, as `MultiRewards` does, and trim the amount so that the amount plus the leftover is a multiple of the duration, keeping the remainder for the next drop.

REMEDIATION COMMENT

Addressed in: 2a90fd02791081b43e8732bf396c365ed2388836

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by having `DropAutomationV2._notify()` read `periodFinish` and `rewardRate` from `MultiRewards.rewardData()` and trim the amount so that `amount + leftover` is a multiple of `rewardsDuration`.

HAL-017 SOLVED

Stock drop deployment does not require the sequencer-guarded price checker before V2 compounding goes live

Informational · 0.8 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

The `017_DeployStockDrop` proposal deploys `MamoStakingRegistryV2` with the live Chainlink price checker proxy (`CHAINLINK_SWAP_CHECKER_PROXY`) as its global price checker, whitelists the `MamoStakingStrategyV2` implementation so that live staking accounts can upgrade to it, and configures the USDC, cbBTC and WETH to MAMO pairs on that checker. `MamoStakingStrategyV2._swapHop()` then derives the minimum output of each hop in `MamoStakingStrategyV2.compound()` from `SlippagePriceChecker.getExpectedOut()` of that checker whenever the route does not name another checker, which covers the cbBTC, WETH and USDC swaps and the USDC to MAMO hop of every stock route. The staking accounts deployed today price their compounds through the Aerodrome quoter, so this proposal makes the Chainlink checker the price source of staking compounds for the first time.

However, the L2 sequencer uptime guard of the price checker only exists in the implementation that the separate `013_SlippagePriceCheckerOracleHardening` proposal installs, and that proposal is also the one that sets the uptime feed and its 3600 second grace period. The checker deployed on Base still runs the previous implementation, which has no sequencer check, and neither `DeployStockDrop.preBuildMock()` nor `DeployStockDrop.validate()` verifies that the uptime feed is configured. If `017_DeployStockDrop` is executed before `013_SlippagePriceCheckerOracleHardening`, the checker keeps quoting during and right after a Base sequencer outage as long as each feed is within its heartbeat, and `MamoStakingStrategyV2.compound()` swaps reward tokens such as cbBTC to MAMO with a minimum output derived from a price that may be stale. When the hardening proposal is executed first, the same quote reverts with "Sequencer is down" during the outage and with "Sequencer grace period not over" for the grace period after it, so the reward token stays on the account and is compounded by a later call.

The exposure is limited to compound calls executed during an outage or before the feeds refresh after it, and to the part of the price movement within the feed heartbeats (3600 seconds for BTC/USD and ETH/USD, 86400 seconds for MAMO/USD and 90000 seconds for USDC/USD) that exceeds the configured slippage. Staked principal is not affected.

RECOMMENDATION

Make `017_DeployStockDrop` depend on the hardened checker. In `DeployStockDrop.preBuildMock()` and `DeployStockDrop.validate()`, assert that `CHAINLINK_SWAP_CHECKER_PROXY` reports a `sequencerUptimeFeed()` equal to `CHAINLINK_L2_SEQUENCER_UPTIME_FEED` and a `sequencerGracePeriod()` of at least 3600 seconds.

Because these checks only run when the proposal is simulated, also enforce the order in the Safe batch itself, for example by adding `SlippagePriceChecker.setSequencerUptimeFeed(CHAINLINK_L2_SEQUENCER_UPTIME_FEED, 3600)` as the first action of `DeployStockDrop.build()`. On the current implementation this call reverts and takes the whole batch with it, and on the hardened implementation it writes the same values that `013_SlippagePriceCheckerOracleHardening` set (adjust them if they were changed afterwards). Execute the hardening proposal before `017_DeployStockDrop` in every fork simulation of the stock drop release, and document this order in the release runbook.

REMEDIATION COMMENT

Addressed in: 7i9bd53fa8cfc56e639c04e4cbbd2971974a81a6

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by making the first action of `DeployStockDrop.build()` a call to `SlippagePriceChecker.setSequencerUptimeFeed(CHAINLINK_L2_SEQUENCER_UPTIME_FEED, 3600)` and asserting the feed in `preBuildMock()` and the feed and grace period in `validate()`.

HAL-018 SOLVED

Order-signer key has no cumulative, rate, count or direction limit

● Informational · 0.7 AO:S/AC:M/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategy::isValidSignature`. One registry-wide order signer co-signs orders for every account. Each order passes only the per-order price check and the deviation band; there is no nonce, rate, count, direction or cumulative-loss limit. `approveCowRelayer` grants the CoW vault relayer an unlimited allowance and any address can call it. A revoked allowance would therefore not hold, and the account has no owner switch that makes `isValidSignature` reject orders.

SOL 241-249

```
241 function approveCowRelayer(address token) external override {
242     if (
243         token != address(asset) && stockRegistry.tokenConfig(token).status ==
    IStockAccountRegistry.TokenStatus.None
244     ) {
245         revert TokenNotListed(token);
246     }
247
248     IERC20(token).forceApprove(cowVaultRelayer, type(uint256).max);
249 }
```

If the order-signer key is compromised or misused, the holder can sign many in-band orders for every account, including alternating round trips that pass `_checkRange` independently because the deviation band and balance checks see a balanced account at each validation time; a solver fills them in successive batches, and each round trip can lose up to the account slippage below the reference (live cap 100 bps, code ceiling 500 bps) before the administrator detects the misuse and rotates the key or pauses the registry. The account owner has no switch that stops order acceptance: `setAccountSlippage(0)` selects the registry cap, so the lowest per-fill tolerance the owner can choose is 1 bps; the other owner options are to leave in kind or to wait for the administrator to rotate the signer or pause the registry. The holder can also pay for one reference manipulation and sign limit-priced orders for many accounts in the same window. Realized loss needs a solver that fills at the limit price; competitive solvers return most surplus to the account.

RECOMMENDATION

Limit what one signed order can do and give owners a durable stop.

1. Add a stateless direction rule in `_checkRange`: reject an order that increases the distance to target of both legs, so that round trips from a balanced state are rejected while rebalancing toward target still passes.
2. Add an owner-only flag in the account, for example `ordersDisabled`, and make `isValidSignature` reject every order while it is set; this stop does not depend on the relayer allowance, which any caller can restore through `approveCowRelayer`, and it leaves the default slippage of existing accounts unchanged.
3. For a cumulative budget, record each order outside the view (for example a backend `registerOrder(digest, sellValue)` that enforces a per-account turnover limit per window) and make `isValidSignature` accept only registered digests. Keep the signer key in an HSM with per-account rate limits in the order service, and monitor fills against the reference.

REMEDIATION COMMENT

Addressed in: c191a8ef18f949ab883bbd8d5e2187d85b99737e

SOLVED: The owner order switch was verified in `StockAccountStrategy.sol:216–220` and `StockAccountStrategy.sol:262–343`. The reported cumulative, rate, count, and direction limits are still absent from the on-chain order path. The stated service controls were not verified in this source review. The following statement was provided by the `Moonwell` team:

> Owner-only setOrdersDisabled(bool): while set, isValidSignature refuses every order (OrdersDisabled), giving the user a stop that does not depend on the relayer allowance. Cumulative, rate and direction limits on the signer key remain operational controls (key in KMS, per-account volume alerts); the per-order value bound (3f2e6ab50e9baa3ab6a4978c880fbe22277a3339) caps what one signed order can move.

HAL-019 SOLVED

TWAP pool accepted without liquidity or observation checks

Informational · 0.6 AO:A/AC:M/AX:H/R:P/S:U/C:N/A:L/I:N/D:M/Y:N

DESCRIPTION

The issue originates in `StockAccountRegistry::_requirePriceable`, where the listing and re-activation probe requires only a non-zero quote. No check of pool liquidity, observation cardinality, or observation age is performed at `listToken`, re-activation, `setPriceChecker`, or `setTwapWindow`. The deployment runbook states that a pool whose history is shorter than `twapWindow` is refused at listing and that only `listToken` decides, through the registry probe; the probe checks only that the quote does not revert and is non-zero.
Runtime `PoolTwap` pricing in `StockAccountPriceChecker::_twapTick` treats any successful return from `pool.observe()` as valid, and nothing requires the pool observation cardinality to cover `twapWindow`.

SOL 299-306

```
299 function _requirePriceable(address token) internal view {
300     uint256 oneToken = 10 ** IERC20Metadata(token).decimals();
301     try priceChecker.getExpectedOut(oneToken, token, asset) returns (uint256 quote) {
            if (quote == 0) revert TokenNotPriceable(token);
302     } catch {
303         revert TokenNotPriceable(token);
304     }
305 }
306
```

When an administrator calls `StockAccountRegistry::listToken` for a pool with cardinality 1 whose single observation is older than `twapWindow`, `_requirePriceable` routes through `StockAccountPriceChecker::_twapTick`, which extrapolates the current tick and returns a non-zero quote, passing the probe and listing the token as `Active`; a pool initialized less than `twapWindow` ago is refused because `observe()` reverts and `_twapTick` maps that to `InsufficientObservations`. A pool accepted with a fresh observation ring quotes spot as the TWAP until a second observation enters the window, and a thin pool can be moved and held across the 180-second window while equity markets are closed, feeding a manipulated price into NAV, `depositToken` principal, fee conversion, order checks, and withdrawal floors; the owner cash withdrawal is the most direct exposure, because a CoW order also needs a backend signature near the floor. A pool listed with too few observation slots can also lose its history to ordinary or deliberate swaps, causing `pool.observe()` to revert and making the token unpriceable for every account, blocking `depositToken`, fee payment in that token, and orders while `withdrawToken` and `withdrawAllInKind` remain available and anyone can grow the cardinality. The four live equity

pricing pools have 2,048 observation slots and hold between 0.8 and 1.45 million USDC each, so no live exposure is shown; listing and pool changes are administrator actions.

RECOMMENDATION

Add pool-quality checks for `PoolTwap` tokens to `StockAccountRegistry::_requirePriceable`, which already runs at listing, re-activation, price-checker change and TWAP-window change. Require an `observationCardinality` that holds at least `twapWindow` of history at the chain block time with a margin, an observation in the ring that is older than `twapWindow`, and a minimum harmonic-mean liquidity over the window, computed from the `secondsPerLiquidityCumulativeX128` values that the same `observe()` call returns. A time-weighted liquidity value is not raised by liquidity added only around the probe, and cardinality cannot decrease, so these checks at the probe points also prevent the later eviction case. Keep the runtime quote path in `StockAccountPriceChecker` unchanged, so that a short spot move cannot remove a token from valuation. Monitor pool liquidity after listing and halt a token whose pool becomes thin.

REMEDIATION COMMENT

Addressed in: 78e818469f03ce9eb0f80ce8812b3b67bf134f1e

SOLVED: The `Moonwell` **team** solved this finding off-chain by making the read-only `make stock-pool-readiness` check (and `make stock-pool-grow` for short pools) a mandatory runbook step before every listing; the contracts are unchanged and nothing on-chain enforces observation history or liquidity, stating:

> The observation-cardinality and liquidity checks stay off-chain and are now a mandatory runbook step before every listing and before signing a listing batch: a read-only make stock-pool-readiness that names short pools and a separate broadcasting make stock-pool-grow. The runbook wording that overstated the on-chain probe is corrected (117d30dad6b9055f52ce65525b2b3b6621fb298e); the on-chain listing probe now quotes both directions (82de9ef80061c40959d434ad5940be563adee22b).

HAL-020 SOLVED

Pinned factory implementation and owner-only upgrades halt rollout

Informational · 0.5 AO:S/AC:L/AX:L/R:P/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

`StockAccountStrategyFactory` stores `strategyImplementation` as an immutable and uses it in the CREATE2 bytecode for every account deployment. `MamoStrategyRegistry::addStrategy` accepts only `latestImplementationById[strategyTypeId]`. When a newer implementation is whitelisted under the same type ID, every `createStrategyForUser` call on the live factory reverts with "Not latest implementation" until a replacement factory is deployed and granted `BACKEND_ROLE`. This behavior is documented in `docs/STRATEGY_REGISTRY_SPEC.md`.

SOL 78–85

```
78  function computeStrategyAddress(address user) public view returns (address) {
79      bytes32 salt = keccak256(abi.encodePacked(user));
80
81      bytes memory bytecode =
82          abi.encodePacked(type(ERC1967Proxy).creationCode,
    abi.encode(strategyImplementation, ""));
83
84      return Create2.computeAddress(salt, keccak256(bytecode));
85  }
```

When a new implementation is whitelisted as the latest under an existing `strategyTypeId`, every call to `StockAccountStrategyFactory::createStrategyForUser` on the existing factory reverts because the immutable `strategyImplementation` no longer matches `latestImplementationById`. Account creation stops for all users until a replacement factory is deployed and its `BACKEND_ROLE` is granted; the predicted address changes with the new factory, and one user can hold two accounts from different factories because the old guard does not cover new factory deployments. Existing accounts are not directly affected and can upgrade individually, but accounts whose owners do not upgrade retain the prior behavior; the registry cannot push new code and no direct loss of funds results.

RECOMMENDATION

Read `MamoStrategyRegistry::latestImplementationById` in `StockAccountStrategyFactory::createStrategyForUser` and `StockAccountStrategyFactory::computeStrategyAddress` to resolve the implementation at creation time, as `MultiMarketStrategyFactory` does. Because the predicted address changes with each implementation, also reject a user who already holds a stock account of this type using

`MamoStrategyRegistry::getUserStrategies`, so that one user cannot hold two accounts. For a pinned-factory design, deploy the replacement factory in the same multisig batch as the whitelist call, revoke `BACKEND_ROLE` from the old factory and add the same duplicate-account check. For existing accounts, plan an owner-facing upgrade campaign and document which fixes require the owner to upgrade, because the registry cannot push new code.

REMEDIATION COMMENT

Addressed in: 2567ad77fcd31d5c30d77b1107fc5dcd7c163216

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by replacing the immutable `strategyImplementation` with a view that reads `mamoStrategyRegistry.latestImplementationById(strategyTypeId)`, used by both `computeStrategyAddress` and `createStrategyForUser`.

HAL-021 SOLVED

Selling down a SellOnly token stops at the ceilings of the receiving legs

Informational · 0.5 AO:S/AC:L/AX:L/R:P/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

When the registry marks a token `SellOnly`, `StockAccountStrategy::_targetOf` returns 0 for that token, removing its sell floor. However, the stored basket weight of the `SellOnly` token is not redistributed to the remaining legs. Each sell-to-cash order therefore increases the cash weight toward its unchanged ceiling, and once that ceiling is reached, `StockAccountStrategy::_checkRange` rejects further sells regardless of their size.

SOL 398-403

```
398 function _targetOf(address token) internal view returns (uint16) {
399     if (token == address(asset)) return cashTargetBps;
400     // A SellOnly token is being wound down, so it is sold toward zero whatever the basket says
401     if (stockRegistry.tokenConfig(token).status == IStockAccountRegistry.TokenStatus.SellOnly) return 0;
402     return _targetBps(token);
403 }
```

When the registry marks a basket token `SellOnly`, the backend co-signs sell orders to convert it into cash. The buy-leg check uses `_targetOf(asset) + maxDeviationBps = cashTargetBps + maxDeviationBps` as the cash ceiling; at the live 1,000 bps band, a basket holding 9,000 bps in stock and 1,000 bps cash reaches the cash ceiling after selling about 2,500 USDC of a 22,500 USDC `SellOnly` holding, and further sells into cash revert with `BuyLeavesTokenAboveRange`. The token cannot be wound down further into cash; sells into an under-weight Active leg stop at that leg's ceiling in the same way. About 80% of the position remains until the owner calls `setBasket`. No funds are lost, and `withdrawToken` or `withdrawAllInKind` are owner workarounds. The code comment states that the token is sold toward zero, which the code does not achieve for a large holding.

RECOMMENDATION

In `_checkRange`, skip the buy-side ceiling check when the sell token is `SellOnly` and the buy token is the asset, so that a `SellOnly` holding can be sold into cash until it reaches zero. Keep every other check as it is, including the cash sell floor and the ceilings of Active buy legs, so that cash-to-stock buys for under-weight legs are not affected.

REMEDIATION COMMENT

Addressed in: f845823db16d35a8de9f484e5683106392ff0fa3

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by skipping the buy ceiling in `_checkRange` through a `windDown` condition when the sell token is `SellOnly` and the buy token is the `asset`.

HAL-022 SOLVED

Missing input validation

Informational · 0.5 AO:S/AC:L/AX:M/R:P/S:U/C:N/A:M/I:N/D:M/Y:M

DESCRIPTION

Throughout the contracts used by the stock drop deployment, there are several functions that assign values to state without validating them. The proposals of this deployment call most of them: `017_DeployStockDrop` calls `MamoStrategyRegistry.whitelistImplementation()` and `SlippagePriceChecker.addTokenConfiguration()`, `006_StockDropCutover` calls `MultiRewards.setRewardsDistributor()`, and `007_AddStockRewards` calls `MultiRewards.addReward()`. `MamoStakingRegistryV2`, deployed by `017_DeployStockDrop`, also exposes an unvalidated setter that its administrator can call to replace the price checker.

Instances of this issue include:

- In `MamoStrategyRegistry.whitelistImplementation()`, `implementation` is only checked against the zero address and an existing entry, not for deployed code or a UUPS `proxiableUUID()`. If an address without code or a proxy address is whitelisted by mistake under an existing type id, it becomes the latest implementation of that type: every `MamoStrategyRegistry.upgradeStrategy()` call for the type reverts because the strategy's UUPS upgrade rejects that address, and every `MamoStrategyRegistry.addStrategy()` call for a strategy of that type reverts, until the administrator whitelists a valid implementation under the same type id. The address stays marked as whitelisted because the contract has no function to remove it.
- In `MultiRewards.setRewardsDistributor()`, `_rewardsToken` is not checked against `rewardTokens`. A distributor set for a token that was never added can set its duration and notify rewards, but `MultiRewards.getReward()` only pays the listed tokens, so the notified amount cannot be claimed by any staker. The owner can recover it with `MultiRewards.recoverERC20()`, except when the token is the staking token. Once a duration is set, `MultiRewards.addReward()` for that token reverts until `MultiRewards.removeReward()` is called after the period ends.
- In `MultiRewards.addReward()`, `_rewardsDuration` is not checked to be non-zero, although a zero duration is also what the function uses to detect an existing entry. Adding the same token twice with a zero duration pushes it twice into `rewardTokens`. After `MultiRewards.removeReward()` deletes one entry and its data, `MultiRewards.earned()` underflows for every staker with a non-zero checkpoint on that token, so their `stake()`, `withdraw()`, `getReward()` and `exit()` calls revert. The remaining entry cannot be removed directly because its data was deleted, so these calls keep reverting until the owner assigns a

distributor for the token, sets a new duration through it, and calls `MultiRewards.removeReward()` again.

- In `SlippagePriceChecker.addTokenConfiguration()`, each feed configuration is only checked for a non-zero feed address and a non-zero heartbeat. The function does not check that the feed answers `latestRoundData()` and `decimals()`, and a feed does not expose on chain which assets it prices, so a route with an inverted `reverse` flag or a feed for another asset is stored without error. `SlippagePriceChecker.getExpectedOut()` then returns a price that can be off by several orders of magnitude, `SlippagePriceChecker.checkPrice()` accepts a minimum output close to zero, and `MamoStakingStrategyV2.compound()` derives its swap minimum from that quote. The `validate()` step of `017_DeployStockDrop` only asserts that each new quote is non-zero, which such a route also passes. A feed whose `decimals()` is 78 or more makes every quote for the pair revert.
- In `MamoStakingRegistryV2.setSlippagePriceChecker()`, the new checker is only checked against the zero address and the current checker. A contract that does not implement `getExpectedOut()` is accepted, after which every `MamoStakingStrategyV2.compound()` call holds each reward token whose route uses the registry's default checker instead of swapping it, until the administrator sets a working checker. In this deployment that is every reward token, because the cbBTC, WETH and USDC routes and the final USDC to MAMO hop of each stock route leave the checker unset. In the same deployment, `MamoStakingStrategyV2.setStakingRegistry()` probes its candidate with static calls before accepting it.

Except for rewards notified in the staking token and reward swaps priced through a misconfigured route, funds are not lost in these cases, but each one requires an administrator action to restore normal operation.

RECOMMENDATION

Consider adding the missing validation:

- In `MamoStrategyRegistry.whitelistImplementation()`, require that `implementation.code.length` is greater than zero and that `proxiableUUID()` returns the ERC-1967 implementation slot.
- In `MultiRewards.setRewardsDistributor()`, require that the token is already listed, for example that its `rewardsDuration` is non-zero.
- In `MultiRewards.addReward()`, require that `_rewardsDuration` is greater than zero.
- In `SlippagePriceChecker.addTokenConfiguration()`, call `latestRoundData()` and `decimals()` on each feed and require a positive answer and a bounded number of decimals. Because a feed does not expose the assets it prices, also assert in the proposals that each new

quote lies within an expected range of an independently computed price, instead of only checking that it is non-zero.

- In `MamoStakingRegistryV2.setSlippagePriceChecker()`, require that the new checker has code and that `isTokenPairConfigured()` returns true for each pair the registry prices through it, such as USDC to MAMO. A storage read keeps this incident remediation setter usable while the sequencer is down or a feed is stale, which a live `getExpectedOut()` probe would not.

As the deployed `MamoStrategyRegistry` and `MultiRewards` are not upgradeable, their checks only apply to new deployments; the `SlippagePriceChecker` checks can ship in a new implementation of its proxy. `MamoStakingRegistryV2` is not deployed yet, so its check can be added before `017_DeployStockDrop` runs. Until then, assert the same conditions in the proposals before each call.

REMEDIATION COMMENT

Addressed in: 8ce6b395e86d9355ee81a5707942e6fa3404d6a6

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by probing `maxTimePriceValid()` on a candidate checker in the `MamoStakingRegistryV2` constructor and `setSlippagePriceChecker()`, and probing `latestRoundData()` and `decimals()` on every feed in `SlippagePriceChecker.addTokenConfiguration()`.

HAL-023 SOLVED

_reducePrincipal pricing differs from NAV for Halted and unpriceable tokens

Informational · 0.4 AO:S/AC:L/AX:M/R:P/S:U/C:N/A:L/I:M/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategy::_reducePrincipal`, which converts a withdrawn token amount to an asset-equivalent value by calling `_tryValue` unconditionally and deducts that value from `principal`. `getNAV()` uses a different branch for Halted tokens: it skips the price check entirely for tokens with `TokenStatus.Halted` and returns zero for unpriceable tokens. Because `_reducePrincipal` does not replicate this exclusion, it can record a non-zero principal reduction when NAV treats the same amount as zero.

SOL 656-660

```
656 function _reducePrincipal(address token, uint256 amount) internal {
657     uint256 value = amount;
658     if (token != address(asset)) (, value) = _tryValue(stockRegistry.priceChecker(),
    token, amount);
659     principal -= Math.min(value, principal);
660 }
```

When the owner calls `withdrawToken` for a token with `TokenStatus.Halted`, NAV excludes the token's balance from the account value, but `_reducePrincipal` calls `_tryValue` and attempts a price check. If the checker still returns a quote, `principal` is decremented by that amount, lowering `principal`, and with it the deposit-cap usage, until a full exit resets it. The management fee is computed from NAV and does not read `principal`, and a later deposit adds its full amount to `principal`. If `_tryValue` fails, it returns zero, so `_reducePrincipal` subtracts nothing; `principal` stays overstated by the value of the withdrawn amount and still counts against `maxStrategyDeposit`, so later deposits to the account can revert with `DepositCapExceeded`. The mismatch persists until a full exit: only `withdrawAll` and `withdrawAllInKind` reset `principal` to zero. Halting is an administrator action used for exceptional circumstances, and the inconsistency requires the specific condition that the halted token is still priceable at the time of withdrawal.

RECOMMENDATION

Apply the NAV pricing rule in `StockAccountStrategy::_reducePrincipal`: treat a Halted token as unpriced, as `_valuation` does, and keep `principal` unchanged for an unpriced withdrawal, which keeps the cap on the strict side until a full exit resets `principal`. Update the `principal` NatSpec to state this rule and the full-exit reset.

REMEDIATION COMMENT

Addressed in: 4ef8dea05b5e469a26d325a8ee362b0474b67157

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by making `_reducePrincipal` return without changing `principal` when the withdrawn token is `TokenStatus.Halted`, matching `_valuation`.

HAL-024 SOLVED

withdrawToken and recoverERC20 revert when the fee consumes the exit token

Informational · 0.4 AO:A/AC:L/AX:M/R:F/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`StockAccountStrategy::withdrawToken` calls `_tryPayFeesBefore(token)` before checking the remaining balance. When the exit token is a listed fee token, `_tryPayFeesBefore` calls `this.payFees(token)`, which pays `min(feeDue, balance)`. If the fee due equals or exceeds the balance, the entire balance is transferred as a partial fee and the post-fee balance is zero. `withdrawToken` then clamps the requested amount to zero and reverts with `ZeroAmount`. The fee payment is a subcall of the same transaction, so this revert also undoes it: the state is unchanged and every retry fails in the same way. `recoverERC20` reaches the same dead end through the same path, where `BaseStrategy::recoverERC20` requires `amount > 0`.

SOL 161-165

```sol
161 function withdrawToken(address token, uint256 amount) external override onlyOwner {
162     _tryPayFeesBefore(token);
163
164     amount = Math.min(amount, IERC20(token).balanceOf(address(this)));
165     if (amount == 0) revert ZeroAmount();
```

When the fee due in the exit token equals or exceeds the account's balance of that token, `_tryPayFeesBefore` transfers the entire balance as a partial fee; `withdrawToken` then clamps the requested amount to zero and reverts with `ZeroAmount`. Because `this.payFees(token)` is a subcall of the same transaction, the outer revert also undoes it, so the state is unchanged and every retry fails in the same way. This condition arises for small positions in large accounts with unpaid fees, for example when a dust balance cannot be sold. `recoverERC20` reaches the same dead end, contradicting the function comment that a fee that cannot be paid stays owed rather than trapping the owner. No value is lost: any address can call `payFees` with the asset, or with another held token, to settle the fee first, which restores the exit path, and `withdrawAllInKind` is unaffected because it calls `payFeesFromAny`.

RECOMMENDATION

In `withdrawToken` and `recoverERC20`, settle the fee from the other holdings first: pay from the asset and then from the other priced listed holdings, and use the exit token only for any remainder after those are used up. Call this settlement through the existing self-call in try/catch, so that a fee that cannot be paid stays owed, and keep the `ZeroAmount` check for a request that is already zero before the fee.

REMEDIATION COMMENT

Addressed in: 1fd727b54c4f1ff6afbea39bb950671c360f5189

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by making `_tryPayFeesBefore` first settle the fee from every other holding through the call to itself `payFeesFromAnyExcept(token)` in try/catch, and only then pay any remainder from the exit token.

HAL-025 ACKNOWLEDGED

Rewards notified while no MAMO is staked are not distributed

Informational · 0.4 AO:A/AC:L/AX:H/R:P/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

The `DropAutomationV2._notify()` function funds each reward stream through `DropAutomationV2.notifyReward()`, which calls `MultiRewards.notifyRewardAmount()`, without checking how much MAMO is staked. In `MultiRewards`, `MultiRewards.rewardPerToken()` returns the stored value while the total staked supply is zero, but the `updateReward` modifier still moves `lastUpdateTime` forward, so the rewards emitted for any time with no stake are credited to no one.

If a drop is notified while nothing is staked, or every staker exits during an active period, the share of each stream for that time stays in `MultiRewards` and is not added to a later period. For the reward tokens other than MAMO the owner can withdraw that share with `MultiRewards.recoverERC20()`, but MAMO rewards cannot be recovered, because MAMO is also the staking token and `MultiRewards.recoverERC20()` rejects it. The live staking pool holds about 154 million staked MAMO, so the condition requires every staker to leave.

RECOMMENDATION

In `DropAutomationV2._notify()`, skip the notification and keep the balance for the next drop when `MultiRewards.totalSupply()` is zero. This does not cover stakers leaving after a stream has started, so also document that rewards emitted while nothing is staked are not redistributed.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> The live staking pool holds about 154 million MAMO, so the condition requires every staker to exit before a drop lands. For reward tokens other than MAMO the owner can recover an unstreamed share with `MultiRewards.recoverERC20()` and return it to `DropAutomationV2` for the next drop.

HAL-026 SOLVED

Deprecated staking factory keeps BACKEND_ROLE after the stock drop cutover

• Informational · 0.4 AO:A/AC:L/AX:H/R:P/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `017_DeployStockDrop` proposal deploys a new `MamoStakingStrategyFactory` wired to `MamoStakingRegistryV2` and the `MamoStakingStrategyV2` implementation. Its build step states that new accounts come only from the new factories, and it grants `BACKEND_ROLE` on `MamoStrategyRegistry` to the new factories while revoking it from `MAMO_STAKING_STRATEGY_FACTORY` and `STOCK_ACCOUNT_STRATEGY_FACTORY`:

SOL

```
// 2. New accounts come only from the new factories
mamoRegistry.grantRole(backendRole, addresses.getAddress(STAKING_FACTORY_KEY));
mamoRegistry.revokeRole(backendRole, addresses.getAddress("MAMO_STAKING_STRATEGY_FACTORY"));
mamoRegistry.grantRole(backendRole, addresses.getAddress(STOCK_FACTORY_KEY));
mamoRegistry.revokeRole(backendRole, addresses.getAddress("STOCK_ACCOUNT_STRATEGY_FACTORY"));
```

However, the older `MAMO_STAKING_STRATEGY_FACTORY_DEPRECATED` factory (`0xd7C3f474761B9A0ca4a8634570E8DCD4f5c92239`) still holds `BACKEND_ROLE` on the deployed `MamoStrategyRegistry`, although the earlier `008_MamoStakingV2Deployment` proposal revokes that role and asserts its removal. Its implementation is still the latest of strategy type 2, and its `stakingRegistry` and `multiRewards` are immutable and point at `MAMO_STAKING_REGISTRY_DEPRECATED` and `MAMO_MULTI_REWARDS_DEPRECATED`. Because `createStrategyForUser()` accepts a call from the user as well as from the backend, any user can still create a registered type-2 staking account for themselves after the cutover. As documented, the guardian's staking registry pause does not stop reward processing on type-2 accounts. The `validate()` function checks the role only for the four factories that the build step changes, so the remaining member is not detected.

As a result, new accounts can still be opened on the retired staking stack, contrary to the stated intent of both proposals. No funds are lost directly, and the owner of such an account can still withdraw.

RECOMMENDATION

Consider revoking `BACKEND_ROLE` from `MAMO_STAKING_STRATEGY_FACTORY_DEPRECATED` in the `017_DeployStockDrop` build step and asserting in `validate()` that it no longer holds the role.

REMEDIATION COMMENT

Addressed in: 705bbf7222ec91e179e83b9420e538eb793d0772

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by adding `mamoRegistry.revokeRole(backendRole, MAMO_STAKING_STRATEGY_FACTORY_DEPRECATED)` to `DeployStockDrop.build()` and asserting in `preBuildMock()` that the role is held and in `validate()` that it is gone.

HAL-027 SOLVED

Stock hop floor priced on a different pool after a pricing pool change

Informational · 0.4 AO:S/AC:L/AX:M/R:P/S:U/C:N/A:M/I:N/D:N/Y:L

DESCRIPTION

The `MamoStakingStrategyV2._swapHop()` function executes each hop of a reward token route on `hop.pool`, which the `017_DeployStockDrop` proposal stores in `MamoStakingRegistryV2` through `_stockRoute()`, and bounds it with a minimum derived from `STOCK_ACCOUNT_PRICE_CHECKER`. That checker computes the TWAP of the pool recorded in `StockAccountRegistry.tokenConfig()`, not of `hop.pool`. At deployment both are the same pool.

However, `StockAccountRegistry.setTokenPool()` lets the `StockAccountRegistry` administrator repoint a halted token to a new pool, and reactivation probes only the new pool. Nothing updates the route in `MamoStakingRegistryV2`, which is administered separately by the F-MAMO Safe, and no proposal or document requires `MamoStakingRegistryV2.setRoute()` to follow a pool change.

As a result, after a repoint without a matching route update, the floor of the stock hop is priced on the new pool while the swap runs on the old one. If the old pool trades below the new pool's TWAP by more than the slippage, every `compound()` call holds that stock reward until the route is updated. The held reward stays in the staking account, and the owner can withdraw it through `MamoStakingStrategyV2.withdrawRewards()`.

RECOMMENDATION

Consider requiring in `MamoStakingStrategyV2._swapHop()` that a stock hop priced by `STOCK_ACCOUNT_PRICE_CHECKER` uses the pool recorded in `StockAccountRegistry.tokenConfig()` for the stock token of the hop, holding the token otherwise. Because the two registries are administered by different Safes, also consider having the F-MAMO Safe update the route with `MamoStakingRegistryV2.setRoute()` while the token is still halted, before it is reactivated after each `StockAccountRegistry.setTokenPool()` call.

REMEDIATION COMMENT

Addressed in: 87b14a50146eebaf8bd1cb812e0774e3fb96d92e

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by adding `require(hop.pool == cfg.pool, "Stock pool mismatch")` to `MamoStakingStrategyV2._checkStockHop()`, so a stock hop only swaps on the pool recorded in `StockAccountRegistry.tokenConfig()`.

HAL-028 ACKNOWLEDGED

Ineffective on-chain slippage floor due to same-transaction pool quote

Informational · 0.3 AO:A/AC:H/AX:H/R:N/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

The `DropAutomationV2._swap()` function executes each Aerodrome CL swap of a drop. Before swapping, it calls `AERODROME_QUOTER.quoteExactInputSingle()` and passes `quoted * (BPS_DENOMINATOR - maxSlippageBps) / BPS_DENOMINATOR` to `AERODROME_CL_ROUTER.exactInputSingle()` as `amountOutMinimum`. The drop documentation in `docs/DROP_AUTOMATION.md`, written for the first version of the contract and still describing the drop flow that `DropAutomationV2` keeps, presents this quote-derived minimum as the MEV protection of the drop swaps, and states that restricting `createDrop()` to the dedicated sender prevents sandwich attacks.

However, the quote is computed against the same pool state, in the same transaction, as the swap it is meant to protect. This holds for both hops of a route through MAMO, because the second hop is quoted after the first one executes. An account that moves the pool price before the `createDrop()` transaction executes shifts the quote and the swap equally, so the quote-derived minimum is always met and `maxSlippageBps` does not bound the execution price against any external reference. Restricting who calls `createDrop()` does not prevent other accounts from trading around that call.

The only floor that is independent of the pool state is the per-token `minAmountOuts_` value supplied by the `dedicatedMsgSender` in `DropAutomationV2.createDrop()` and enforced on the final cbBTC output in `DropAutomationV2._swapToCbBtc()`. Its effectiveness depends entirely on how the off-chain automation computes it. The `DropAutomationV2` NatSpec describes it only as the minimum cbBTC each token must return, without requiring it to be derived from an independent price or to be nonzero. The guidance present in the first version of the contract, that these minimums be calculated off-chain from fair market prices, was not carried over, and the drop documentation does not mention the parameter.

If the automation submits a zero or loose `minAmountOuts_` value, an account able to move the pool price ahead of the drop and restore it afterwards, for example by anticipating the weekly drop schedule, can capture the difference between that value and the fair price. This reduces the cbBTC streamed to MultiRewards stakers in that drop.

RECOMMENDATION

Require the off-chain automation to compute each `minAmountOuts_` entry in cbBTC for the full route from a price that does not depend on the current state of the swapped pools, such as Chainlink feeds

or a TWAP over a window longer than the drop transaction, with an explicit tolerance. Leave out of `swapTokens_` any token that has no reliable independent price or whose balance is too small to swap, so every submitted minimum is nonzero. Additionally, document this requirement in the `DropAutomationV2.createDrop()` NatSpec, and rewrite the MEV protection sections of `docs/DROP_AUTOMATION.md` to identify `minAmountOuts_` as the only price bound of a drop that is independent of the pool state.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> The quote-derived minimum is a sanity bound, not the MEV floor. The independent floor is the per-token `minAmountOuts_` that the dedicated sender supplies to `createDrop()` and that `_swapToCbBtc()` enforces on the final `cbBTC` output; the dedicated sender's automation is required to derive it from an external price reference (Chainlink / TWAP), never from the pool, and `createDrop()` is restricted to that sender. Drops are small relative to pool depth and the exposure is bounded to the difference between that off-chain floor and the fair price for one drop. Kept as an operational control of the automation; no on-chain change.

HAL-029 ACKNOWLEDGED

Same-batch orders validated against the same pre-trade state

Informational · 0.3 AO:S/AC:L/AX:M/R:P/S:U/C:N/A:N/I:M/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategy::isValidSignature`, which is declared `view` and can therefore record no pending-order state. It checks each order independently against the live balances and `_valuation()` without any nonce or record of other accepted orders. CoW settlement validates all signatures in a batch before transferring any token, so several co-signed orders for the same account in one batch each pass the balance and band checks in isolation while their combined effect is not bounded.

SOL 305-316

```
305         if (order.sellAmount > IERC20(sellToken).balanceOf(address(this))) revert SellExceedsBalance();
306
307         uint256 expectedOut = stockRegistry.priceChecker().getExpectedOut(order.sellAmount, sellToken, buyToken);
308         if (expectedOut == 0) revert PriceCheckFailed();
309
310         uint256 slip = getAccountSlippage();
311         if (order.buyAmount < (expectedOut * (TOTAL_BPS - slip)) / TOTAL_BPS) revert PriceCheckFailed();
312
313         _checkRange(sellToken, buyToken, order.sellAmount, order.buyAmount, expectedOut);
314         return MAGIC_VALUE;
315     }
316
```

When the order signer co-signs two orders for the same account in the same settlement period, for example order A sells 40% of the cash leg for token X and order B sells another 40% for token Y, CoW settlement validates both signatures against the pre-batch state. Each call to `isValidSignature` sees the unchanged account balances, so both signatures return `MAGIC_VALUE` and settlement transfers both sell amounts. The post-settlement cash weight ends below any reasonable floor, but no on-chain check bounded the combined effect. Each order is still limited by the pre-trade balance of its sell token and by the per-order price check, so with fair prices the result is a basket outside the user's targets rather than a direct loss. A later rebalance can correct the weights, and the condition requires the order signer to co-sign overlapping orders and a solver to include them in one batch.

RECOMMENDATION

Prevent concurrent orders for one account, or bound them. Because `isValidSignature` is a view and cannot record state, an on-chain control needs a state update outside it: for example, a backend-

called `setActiveOrder(bytes32 digest)` that stores the single active order digest per account, and a check in `isValidSignature` that `orderDigest` equals that digest. If an on-chain control is not adopted, enforce one open order per account in the order service and document it as a trust assumption of the order signer.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> Same-batch orders are validated against pre-trade state by construction of EIP-1271 settlement. Operational control, in force since the 2026-09-22 review: the backend planner emits at most one order per account per `CoW` auction (backend spec, sizing section), so no two orders on one account settle in the same batch.

HAL-030 SOLVED

Account ownership transfer desyncs factory per-user creation state

Informational · 0.3 AO:S/AC:L/AX:L/R:P/S:U/C:N/A:L/I:L/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategyFactory::createStrategyForUser`. The factory derives each account's CREATE2 salt from `keccak256(abi.encodePacked(user))`, binding the address to the original creator rather than the current owner. `BaseStrategy::transferOwnership` moves the registry entry through `MamoStrategyRegistry.updateStrategyOwner` but does not update the factory's slot. After a transfer, the previous owner's salt slot remains occupied, the new owner's `computeStrategyAddress` returns a different predicted address than the transferred account, and the new owner can deploy a second account at that predicted address.

SOL 78–85

```
78      function computeStrategyAddress(address user) public view returns (address) {
79          bytes32 salt = keccak256(abi.encodePacked(user));
80
81          bytes memory bytecode =
82              abi.encodePacked(type(ERC1967Proxy).creationCode,
    abi.encode(strategyImplementation, ""));
83
84          return Create2.computeAddress(salt, keccak256(bytecode));
85      }
```

When User A deploys an account, the factory records code at the `keccak256(A)`-derived address. If User A transfers the account to User B through `transferOwnership`, the registry ownership moves but the factory salt slot for A still shows code, so any subsequent call to `createStrategyForUser(A, ...)` triggers `StrategyAlreadyExists` and User A cannot open a new account. User B's call to `computeStrategyAddress(B)` returns a fresh, undeployed address that differs from the account transferred to B, and User B can deploy a second, unrelated account at that address. No funds are at risk; the effects are an onboarding lockout for the original user and an address-derivation inconsistency for off-chain tooling.

RECOMMENDATION

Bind the duplicate-account check to current ownership. In `createStrategyForUser`, revert when `MamoStrategyRegistry::getUserStrategies(user)` already contains an account whose `strategyTypeId` equals the factory's `strategyTypeId`. Derive the CREATE2 salt from the user and a per-user nonce that the factory increments on each creation, and give `computeStrategyAddress` the

same nonce input, so that a previous owner can open a new account. Have off-chain tooling resolve accounts through `MamoStrategyRegistry` ownership.

REMEDIATION COMMENT

Addressed in: 406ec7247550a340db0cfb5c879a164016a491cc

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by making `createStrategyForUser` revert with `StrategyAlreadyExists` when `MamoStrategyRegistry::getUserStrategies(user)` already holds a strategy with the factory's `strategyTypeId`.

HAL-031 SOLVED

Slippage and deviation caps accept zero

Informational · 0.3 AO:S/AC:L/AX:M/R:F/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountRegistry::_setMaxDeviationBps`, `_setMaxBackendSlippageBps`, and `_setMaxWithdrawSlippageBps`, which each enforce only an upper bound and accept 0, or any value below the pool fee tier, without reverting, while `_setMaxPositions` and `_setMinTargetBps` reject 0. These setters are restricted to `DEFAULT_ADMIN_ROLE`, and every account reads the caps live.

`maxDeviationBps` is the band around each basket target weight that an order may leave a token in; at 0 the band is zero, but orders that move both legs toward their targets without overshooting them still pass. A zero `maxBackendSlippageBps` makes `isValidSignature` reject an order whose `buyAmount` is below the reference; fills at or above the reference still pass. A zero `maxWithdrawSlippageBps`, or any value below the pool fee tier, sets every withdrawal sell minimum at or near the full fee-free reference value, so every `withdraw` or `withdrawAll` that has to sell reverts.

SOL 346-371

```sol
346  function _setMaxDeviationBps(uint16 newMaxDeviationBps) internal {
347      if (newMaxDeviationBps > 10_000) revert InvalidMaxDeviation();
348
349      uint16 oldValue = maxDeviationBps;
350      maxDeviationBps = newMaxDeviationBps;
351
352      emit MaxDeviationBpsUpdated(oldValue, newMaxDeviationBps);
353  }
354
355  function _setMaxBackendSlippageBps(uint16 newSlippageBps) internal {
356      if (newSlippageBps > backendSlippageCeilingBps) revert InvalidSlippageCap();
357
358      uint16 oldValue = maxBackendSlippageBps;
359      maxBackendSlippageBps = newSlippageBps;
360
361      emit MaxBackendSlippageBpsUpdated(oldValue, newSlippageBps);
362  }
363
364  function _setMaxWithdrawSlippageBps(uint16 newSlippageBps) internal {
365      if (newSlippageBps > withdrawSlippageCeilingBps) revert InvalidSlippageCap();
366
367      uint16 oldValue = maxWithdrawSlippageBps;
368      maxWithdrawSlippageBps = newSlippageBps;
369
370      emit MaxWithdrawSlippageBpsUpdated(oldValue, newSlippageBps);
371  }
```

The registry administrator (`DEFAULT_ADMIN_ROLE`) sets `maxWithdrawSlippageBps` to 0, or to a value below the pool fee tier, for example as an emergency setting; the setter stores the value without error. An account owner then calls `withdraw` for an amount above the idle USDC balance, or `withdrawAll` while holding a stock position. Each sell leg must return the full TWAP reference value, while the Aerodrome pool charges a 0.05% fee and the TWAP reference has no fee term, so the sell leg reverts unless spot sits above the TWAP by more than the pool fee. The withdrawal reverts for every account until the administrator sets a working value. Withdrawals paid from idle USDC, `withdrawToken`, `recoverERC20` and `withdrawAllInKind` still work. A zero `maxBackendSlippageBps` or `maxDeviationBps` narrows which orders pass but does not stop trading. The deployment configuration sets non-zero caps, and the deployment proposal asserts that the registry holds the configured values, so the risk is a misconfiguration after deployment.

RECOMMENDATION

Add a lower bound in `StockAccountRegistry::_setMaxWithdrawSlippageBps`: a constant floor above the highest fee tier of the pools the registry lists, plus a margin for normal TWAP-to-spot drift. Add a non-zero minimum in `_setMaxBackendSlippageBps` and `_setMaxDeviationBps` as well, as `_setMaxPositions` and `_setMinTargetBps` do, so that every cap keeps a usable tolerance.

REMEDIATION COMMENT

Addressed in: 29f804a8915143b6e6a727074e6894a2a22aba67

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by adding lower bounds in `StockAccountRegistry`: `_setMaxWithdrawSlippageBps` rejects values below 50 bps, `_setMaxBackendSlippageBps` below 10 bps and `_setMaxDeviationBps` below 100 bps.

HAL-032 ACKNOWLEDGED

Unguarded fee transfer lets a blocked fee recipient stop cash exits and deposits

● Informational · 0.2 AO:S/AC:L/AX:M/R:F/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategy::_payFees`, which uses `safeTransfer` to `feeRecipient` without any error handling. `withdraw` and `withdrawAll` call `_payFees(asset)` directly, and `deposit` and `depositToken` reach the same path through `_settleFeesBeforeDeposit` when the account holds USDC. If USDC blacklists the fee recipient, the transfer reverts and all four functions revert for every account that uses that recipient.

SOL 439-448

```
439  function withdrawAll(uint16 maxSlippageBps) external override onlyOwner {
440      _checkWithdrawSlippage(maxSlippageBps);
441
442      (address[] memory tokens, uint256[] memory balances, uint256[] memory values) =
     _sellable();
443      uint256 sold = _executeSells(tokens, balances, values, maxSlippageBps);
444
445      _payFees(address(asset));
446
447      uint256 usdcOut = asset.balanceOf(address(this));
448      if (usdcOut == 0) revert EmptyBalance();
```

When a backend call sets `feeRecipient` to an address that USDC subsequently blacklists, or USDC directly blacklists the factory-seeded recipient, a due fee becomes non-zero after about one second at a 25,000 USDC value and 100 bps. The next call to `withdraw`, `withdrawAll`, `deposit` or `depositToken` triggers `_payFees(asset)`, which reverts on the blocked transfer; all four functions then revert for every account sharing that recipient. The factory seeds every account with the same initial recipient, so a single blacklist decision affects all accounts. Funds are not frozen because `withdrawToken` and `withdrawAllInKind` isolate the fee and still work, but the owner cannot convert positions to USDC or add funds until a backend call rotates `feeRecipient` on each affected account.

RECOMMENDATION

Record an unpaid fee and let the call continue. Add an `accruedFee` storage amount in asset units. In `_payFees`, send the fee with the checked low-level call that `_sendAllInKind` already uses; when the transfer fails, add the due to `accruedFee` and advance `lastFeePaid`, so that deposits and cash exits proceed and new deposits are not charged for earlier time. Collect `accruedFee` first in the next

successful payment, and include it in the `_shortfall` sizing of `withdraw`. In `withdrawAll`, keep `accruedFee` of the asset in the account and send the remainder to the owner.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> A blocked fee recipient needs either the F-MAMO Safe to be blacklisted by USDC or a bad recipient set through the finding above, which is now admin-only in the registry. In-kind exits (`withdrawToken`, `withdrawAllInKind`) keep working regardless. An `accruedFee` fallback would add more code than the residual risk warrants.

HAL-033 SOLVED

Reverting balanceOf of a listed token blocks valuation

Informational · 0.2 AO:S/AC:L/AX:H/R:P/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

`StockAccountStrategy::_valuation`, `_sellable`, `heldTokens` and `_sendAllInKind` iterate over the registry's append-only token list and call `balanceOf` on each token without isolation. In `_valuation` and `_sellable`, the balance read occurs before the `Halted` status check, so marking a token `Halted` does not prevent the revert from propagating. If a listed token's `balanceOf` reverts after an issuer upgrade, every downstream function that calls `_valuation` reverts across all accounts.

SOL 350-358

```
350  for (uint256 i = 0; i < tokens.length; i++) {
351      uint256 balance = IERC20(tokens[i]).balanceOf(address(this));
352      if (balance == 0) {
353          continue;
354      }
355
356      if (stockRegistry.tokenConfig(tokens[i]).status ==
     IStockAccountRegistry.TokenStatus.Halted) {
357          continue;
358      }
```

If a listed token's `balanceOf` reverts after an issuer upgrade, the failure propagates through `StockAccountStrategy::_valuation` and blocks `deposit`, `depositToken`, `withdraw`, `withdrawAll`, `payFees`, `isValidSignature` and `withdrawAllInKind` for every account that holds or iterates the affected token. Halting the token in the registry provides no protection because the balance read occurs before the status check, so the guard cannot intercept the revert. No funds are lost; `StockAccountStrategy::withdrawToken` remains available per token, and owners can recover individual holdings while a fix is deployed. The trigger requires the token issuer to change behavior, and no reverting `balanceOf` is known for the five tokens currently listed.

RECOMMENDATION

Move the `Halted` check ahead of the `balanceOf` call in `StockAccountStrategy::_valuation` and `StockAccountStrategy::_sellable`, and read balances with a gas-limited try/catch in `_valuation`, `_sellable`, `StockAccountStrategy::heldTokens` and `StockAccountStrategy::_sendAllInKind`, treating a failed read as an unpriced or skipped holding. Keep `StockAccountStrategy::withdrawToken` reading only the named token.

REMEDIATION COMMENT

Addressed in: b3dec9728fc2a981de1b5276cdfa9f175807db73

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by moving the `Halted` check ahead of the balance read in `_valuation` and `_sellable` and routing every remaining balance read of the listed set through `_tryBalance`, which treats a reverting or codeless token as unreadable.

HAL-034 SOLVED

Fee post-hook and valuation gas grow with the listed-token count, which has no bound

Informational · 0.2 AO:A/AC:L/AX:H/R:F/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The CoW settlement calls the fee post-hook with a fixed `HOOK_GAS_LIMIT` of 1,000,000 gas. The hook calls `payFees`, which drives `_feeDue` through `getNAV` and then `_valuation`, iterating over every listed token and pricing every non-zero, non-Halted balance. `StockAccountRegistry::listToken` appends to the token list with no maximum count check, and the registry has no removal path, so the gas consumed by `_valuation` grows without bound as more tokens are listed.

SOL 202-214

```sol
function listToken(address token, TokenConfig calldata cfg) external
onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    if (token == asset) revert AssetNotListable();
    if (_tokenConfig[token].status != TokenStatus.None) revert TokenAlreadyListed(token);
    if (cfg.status != TokenStatus.Active) revert MustListAsActive();
    // ... validation checks ...

    _tokenConfig[token] = cfg;
    _tokens.push(token);

    _requirePriceable(token);

    emit TokenListed(token, cfg);
}
```

The system currently lists 5 tokens, and the runbook documents about 667,000 gas for 10 held holdings, leaving a margin of about 333,000 gas. Any address can donate dust of each listed token to a target account to maximize the number of priced holdings in `_valuation`. As the administrator lists additional tokens, an account eventually reaches about 15 to 16 priced holdings, at which point the gas consumed by `_valuation` in the post-hook exceeds 1,000,000. The hook fails silently; the order still settles on CoW, and the fee stays owed until the next uncapped path collects it. The risk grows permanently with each listing because the token list has no removal path and no count check.

RECOMMENDATION

Add a maximum listed-token count to `StockAccountRegistry` and check it in `listToken`. Set it from a gas measurement of `payFees` with every listed token held (for example 12 tokens for the 1,000,000 gas hook budget, with margin), and add a test that runs the post-hook at that limit. `HOOK_GAS_LIMIT` is a constant of each account implementation and each owner upgrades separately, so size the cap for the lowest hook gas limit of any implementation that live accounts still use.

REMEDIATION COMMENT

Addressed in: c6b9fd887606c18ca75c0bc9ab888d736dfde7d0

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by raising `HOOK_GAS_LIMIT` to 4,200,000 so it covers the measured cost of `payFees` with every token held at the new `MAX_LISTED_TOKENS = 30` cap enforced in `listToken`, adding `delistToken` for Halted tokens and a Base fork test that asserts the cap fits the hook budget.

HAL-035 SOLVED

Pricing pool is not checked against the router's factory

Informational · 0.2 AO:S/AC:L/AX:M/R:F/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountRegistry::_requireAssetPool`, which is the only on-chain validation applied when a pool is recorded at listing or re-pointing. The check confirms only that the pool's `token0` and `token1` match the token and the asset; it does not verify that the pool belongs to the router's factory. `_sell` passes the recorded pool's `tickSpacing` to `exactInputSingle`, and the router resolves the execution pool from its own factory by `(tokenIn, tokenOut, tickSpacing)`, so the TWAP reference and the execution venue can differ after an administrator change that pairs a pool and a router from different factories.

SOL 294-297

```
294     function _requireAssetPool(address token, address pool) internal view {
295         (address a, address b) = (ICLPool(pool).token0(), ICLPool(pool).token1());
296         if (!(a == token && b == asset) && !(a == asset && b == token)) revert
    PoolNotAgainstAsset(pool);
297     }
```

An administrator can call `listToken` or `setTokenPool` with a pool from a factory that differs from the current `aerodromeRouter`'s factory, or call `setAerodromeRouter` to a router backed by a different factory without re-validating existing pools; `_requireAssetPool` passes because `token0` and `token1` match. At settlement or withdrawal, `StockAccountStrategy::_sell` reads `tickSpacing` from the recorded pricing pool and supplies it to the router's `exactInputSingle`; if no such pool exists in that factory, the call reverts, and if a different pool exists there, execution proceeds on a venue that did not supply the floor price. The router still enforces the minimum output, so the loss is bounded by the slippage tolerance, but the reference price comes from a venue the router does not use. The live configuration is consistent: all five recorded pools belong to the current router's factory.

RECOMMENDATION

Bind each recorded pool to the router's factory for every price source, because `_sell` reads `tickSpacing` from the recorded pool for every token. Require `ICLPool(pool).factory() == router.factory()` and `ICLFactory(router.factory()).getPool(token0, token1, ICLPool(pool).tickSpacing()) == pool` in `listToken`, `setTokenPool` and every status loosening in `setTokenStatus`. In `setAerodromeRouter`, run the same check for every token that is not Halted. A move to a new factory then stays possible: halt the affected tokens, change the router, repoint each pool with `setTokenPool` and reactivate the tokens.

REMEDIATION COMMENT

Addressed in: f8de526fcb0137f2592072a3570f50318ca80e24

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by requiring `ICLPool(pool).factory() == aerodromeRouter.factory()` in `_requireAssetPool`, which runs in `listToken` and `setTokenPool`.

HAL-036 SOLVED

maxTimePriceValid returns the TWAP window for unlisted tokens

Informational · 0.2 AO:A/AC:L/AX:H/R:F/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountPriceChecker::maxTimePriceValid`. For a token whose registry status is `None` (not listed), the Chainlink delegation branch does not apply and the function returns `registry.twapWindow()`, a non-zero value. However, every pricing call for a `None`-status token reverts with `TokenNotListed`, and the `SlippagePriceChecker` implementation of the same interface returns 0 for an unconfigured token. The two implementations therefore disagree on the meaning of `maxTimePriceValid` for a token that cannot be priced.

SOL 95-106

```
95      function maxTimePriceValid(address token) external view override returns (uint256) {
            if (token != quoteAsset) {
96              IStockAccountRegistry.TokenConfig memory cfg = registry.tokenConfig(token);
97              if (
                    cfg.status != IStockAccountRegistry.TokenStatus.None
98                      && cfg.source == IStockAccountRegistry.PriceSource.Chainlink
99              ) {
100                 return existingChecker.maxTimePriceValid(token);
101             }
102         }
103         return registry.twapWindow();
104     }
105
```

When a caller queries `maxTimePriceValid` on `StockAccountPriceChecker` for a token with registry status `None`, the `Chainlink` branch does not match and the function returns `registry.twapWindow()` rather than 0. A caller that follows the `SlippagePriceChecker` meaning of the return value, where 0 indicates that a token cannot settle, would take a different branch. No contract in the repository reads this value from `StockAccountPriceChecker` directly, so the effect is an interface inconsistency for an integrating system rather than an on-chain exploitable path.

RECOMMENDATION

In `maxTimePriceValid`, inside the existing `token != quoteAsset` branch, return 0 when the registry status of the token is `None`, matching `SlippagePriceChecker`. Keep the `twapWindow` result for the quote asset and for listed `PoolTwap` tokens.

REMEDIATION COMMENT

Addressed in: aafd8b8157cc454c81ac051bac04aada64a0090e

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by making `StockAccountPriceChecker::maxTimePriceValid` return `0` when the registry status of a token other than the quote asset is `TokenStatus.None`.

HAL-037 SOLVED

setPriceChecker probe covers only the token-to-asset direction

Informational · 0.2 AO:S/AC:L/AX:M/R:F/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountRegistry::_requirePriceable`, which quotes each token only with `getExpectedOut(oneToken, token, asset)` and is the sole acceptance check used at `setPriceChecker`, `setTwapWindow`, `listToken` and every status loosening. Accounts also need asset-to-token quotes for fee conversion and buy legs, and token-to-token quotes for stock-to-stock orders, but neither direction is probed when a replacement checker is installed or when a token is listed.

SOL 299–306

```
299 function _requirePriceable(address token) internal view {
300     uint256 oneToken = 10 ** IERC20Metadata(token).decimals();
301     try priceChecker.getExpectedOut(oneToken, token, asset) returns (uint256 quote) {
            if (quote == 0) revert TokenNotPriceable(token);
302     } catch {
303         revert TokenNotPriceable(token);
304     }
305 }
306
```

An administrator installs a replacement `ISlippagePriceChecker` that correctly quotes every token in the token-to-asset direction but does not implement the asset-to-token or token-to-token directions. `_requirePriceable` loops over every non-halted token, calls `getExpectedOut(oneToken, token, asset)`, all probes pass, and the new checker is installed. After installation, `StockAccountStrategy::_feeAmount` calls `getExpectedOut(due, asset, token)` to convert the fee due into the chosen token; the checker reverts on this direction. Every `deposit` and `depositToken` call that triggers `_settleFeesBeforeDeposit` and falls back to a token fee payment reverts, and `isValidSignature` also reverts for buy orders and stock-to-stock orders. Valuation and withdrawal sells use only the token-to-asset direction, and the in-kind exits wrap fee payment, so exits keep working. The production `StockAccountPriceChecker` derives every direction from the same one-token quote, so the gap matters only for a different replacement checker. The effect needs an administrator to install such a checker and is reversed by restoring the previous one.

RECOMMENDATION

Extend `StockAccountRegistry::_requirePriceable` to the quote directions that accounts use. For the probed token, also require a non-zero `getExpectedOut(oneAsset, asset, token)`. When another listed token is not Halted, also require a non-zero token-to-token quote between the probed

token and that token, taking the partner only from tokens that are not Halted. A Halted token whose quote reverts then cannot block `setPriceChecker`, `setTwapWindow`, `listToken` or a status loosening. One change in this shared function covers all four entry points.

REMEDIATION COMMENT

Addressed in: 82de9ef80061c40959d434ad5940be563adee22b

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by making `StockAccountRegistry::_requirePriceable` probe both `token` to `asset` and `asset` to `token` through a new `_requireQuote` helper.

HAL-038 SOLVED

No cross-check of the factory, registry and price checker assets

Informational · 0.2 AO:S/AC:L/AX:H/R:P/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategyFactory::constructor` and `StockAccountPriceChecker::constructor`. Both accept their respective `asset` and `quoteAsset` immutables with only a zero-address check; neither verifies that the value matches `StockAccountRegistry::asset`. A factory deployed with a mismatched asset creates accounts silently: those accounts hold a different unit of account, which is either treated as a listed token or falls outside valuation entirely. A checker mismatch is caught only when a listing probe reverts because no token can price against a different quote asset.
`StockAccountRegistry::_setPriceChecker` checks only that the checker has contract code. Proposal 016 asserts the three agree for the deployed system, but the on-chain constructors provide no permanent guard for future deployments.

SOL 36–43

```
36      constructor(IStockAccountRegistry registry_, address quoteAsset_,
        ISlippagePriceChecker existingChecker_) {
37          if (address(registry_) == address(0) || quoteAsset_ == address(0) ||
        address(existingChecker_) == address(0)) {
38              revert ZeroAddress();
39          }
40          registry = registry_;
41          quoteAsset = quoteAsset_;
42          existingChecker = existingChecker_;
43      }
```

If a new `StockAccountStrategyFactory` is deployed with an `asset` that differs from the registry's `asset`, accounts created by that factory hold an unrecognized unit of account; if the wrong asset is itself a listed token, valuation is distorted rather than reverting. A mismatched price checker fails closed at listing time because `StockAccountRegistry::_requirePriceable` cannot compose quotes against a different quote asset, but the factory case has no equivalent guard and produces accounts silently. The initial deployed system is consistent because the proposal assertions verify that all three agree; the gap applies only to future factory or checker deployments that bypass those assertions.

RECOMMENDATION

In the factory constructor, require that the `_asset` parameter equals `StockAccountRegistry::asset`. In `StockAccountRegistry::_setPriceChecker`, where the new

checker exposes a `quoteAsset` function, require that the returned value equals `asset`. Preserve the deployment proposal assertions.

REMEDIATION COMMENT

Addressed in: 2c9a6a84f32a85f9a2115084d812befc656657cc

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by requiring `_asset == IStockAccountRegistry(_stockRegistry).asset()` in the `StockAccountStrategyFactory` constructor and rejecting, in `StockAccountRegistry::_setPriceChecker`, any checker whose `quoteAsset()` differs from `asset`.

HAL-039 SOLVED

Cached CoW domain separator and relayer not validated and never re-synced

Informational · 0.2 AO:S/AC:L/AX:H/R:P/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategy::initialize`, which reads `domainSeparator()` and `vaultRelayer()` from the settlement contract once and stores them in `cowDomainSeparator` and `cowVaultRelayer` without checking the returned values. No setter or reinitializer updates them, so an account stays bound to the settlement address provided at initialization until its owner upgrades it to a new implementation.

SOL 101-119

```
101 function initialize(InitParams calldata params) external initializer {
102     if (params.asset == address(0)) revert ZeroAddress();
103     if (params.cowSettlement == address(0)) revert ZeroAddress();
104     if (params.feeRecipient == address(0)) revert ZeroAddress();
105     if (params.mamoStrategyRegistry == address(0)) revert ZeroAddress();
106     if (params.stockRegistry == address(0)) revert ZeroAddress();
107     if (params.strategyTypeId == 0) revert StrategyTypeIdNotSet();
108
109     __BaseStrategy_init(params.mamoStrategyRegistry, params.strategyTypeId,
    params.owner);
110
111     asset = IERC20(params.asset);
112     stockRegistry = IStockAccountRegistry(params.stockRegistry);
113     cowDomainSeparator = IGPv2Settlement(params.cowSettlement).domainSeparator();
114     cowVaultRelayer = IGPv2Settlement(params.cowSettlement).vaultRelayer();
115     feeRecipient = params.feeRecipient;
116     lastFeePaid = uint64(block.timestamp);
117
118     _setBasket(params.entries, params.cashTargetBps);
119 }
```

If CoW Protocol moves order settlement to a new contract or relayer, or a factory is deployed with a wrong settlement address, the affected accounts cannot validate CoW orders or approve the correct relayer, so CoW rebalancing and the fee post-hook stop. Recovery needs a new implementation and an upgrade that only each account owner can start, so accounts whose owners do not act stay on the old settlement. The old relayer keeps its allowance, which is usable only through the old settlement with a valid backend-signed order. Owner exits do not depend on these values. The deployment proposal asserts that the deployed factory uses the configured settlement address.

RECOMMENDATION

Add checks in `StockAccountStrategy::initialize` that `cowDomainSeparator` is non-zero and `cowVaultRelayer` is a non-zero address. Keep migration through owner upgrades, and prepare for a

CoW migration: an implementation that reads the new settlement values and revokes the old relayer allowance, and a plan to prompt owners to upgrade.

REMEDIATION COMMENT

Addressed in: 93a2b40d79b82c02a5f52a07d40312ae0289b73e

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by making `initialize` revert with `InvalidCowSettlement` when the settlement returns a zero `domainSeparator()` or a zero `vaultRelayer()`.

HAL-040 SOLVED

TWAP window accepts any nonzero value

Informational · 0.2 AO:S/AC:L/AX:H/R:P/S:U/C:N/A:N/I:M/D:L/Y:N

DESCRIPTION

`StockAccountRegistry::_setTwapWindow` rejects only a zero value, so the administrator can set any nonzero `uint32` window. A very short window makes PoolTwap references follow the manipulable spot tick instead of a time-averaged price. A very long window passes the one-time re-probe that `setTwapWindow` performs on non-halted tokens, but can later cause `observe` to revert when the pool's observation ring no longer covers the requested period, which drops the affected token from account valuations.

SOL 373-378

```
373     function _setTwapWindow(uint32 newTwapWindow) internal {
374         if (newTwapWindow == 0) revert InvalidTwapWindow();
375         uint32 oldValue = twapWindow;
376         twapWindow = newTwapWindow;
377         emit TwapWindowUpdated(oldValue, newTwapWindow);
378     }
```

If the administrator sets a very short window, such as 1 second, `_setTwapWindow` accepts it and the one-time re-probe succeeds because a one-second observation is almost always available. Subsequent calls to `StockAccountPriceChecker::_twapTick` pass that window to `observe([1, 0])`, and the returned tick follows the current pool price with a near-zero lag; one or two blocks of trading can move it materially. Valuation, `depositToken` principal, fee conversion, order floors and withdraw floors for `PoolTwap` tokens then use the near-spot reference, and the documented manipulation-cost guarantee no longer holds. A window longer than the pool's observation ring will eventually cover causes `observe` to revert; the price checker maps the revert to `InsufficientObservations`, the token drops from account NAV, and orders that touch it revert. PROD is configured at 180 seconds; the risk requires an unsafe administrator action.

RECOMMENDATION

Add `MIN_TWAP_WINDOW` and `MAX_TWAP_WINDOW` constants to `StockAccountRegistry` and make `_setTwapWindow` revert outside them. Take the minimum from the documented manipulation-cost analysis (for example the 180-second production value). Set the maximum as a fixed sanity ceiling (for example one hour). The time that a pool's observations cover depends on its observation cardinality and trade frequency, which change over time, so keep the re-probe in `setTwapWindow` and run the pool readiness check for every listed pool before a window change.

REMEDIATION COMMENT

Addressed in: 45196ba66deb6c61ef09258e4579300ceaab211c

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by adding `MIN_TWAP_WINDOW = 60` and `MAX_TWAP_WINDOW = 3600` to `StockAccountRegistry` and making `_setTwapWindow` revert `InvalidTwapWindow` outside that range.

HAL-041 ACKNOWLEDGED

Range check uses the signed buy amount and checks only the two trade legs

● Informational · 0.2 AO:S/AC:L/AX:M/R:P/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`StockAccountStrategy::_checkRange` bounds the buy weight with the signed `buyAmount`, or with `expectedOut` when that is larger, rather than the amount the CoW settlement delivers. `isValidSignature` accepts only fill-or-kill sell orders, which can settle with solver surplus above `buyAmount`. When solver surplus is present, the actual post-settlement buy weight can exceed `target + maxDeviationBps` by about the size of the surplus, violating the documented guarantee that every accepted order leaves all tokens within their bands.

SOL 318-341

```sol
function _checkRange(
    address sellToken,
    address buyToken,
    uint256 sellAmount,
    uint256 buyAmount,
    uint256 expectedOut
) internal view {
    (address[] memory tokens, uint256[] memory values, uint256 nav) = _valuation();

    uint256 sellHeld = _heldValue(tokens, values, sellToken);
    uint256 available = IERC20(sellToken).balanceOf(address(this));
    uint256 sellValue = sellToken == address(asset) ? sellAmount : (sellHeld * sellAmount) / available;
    uint256 buyValue = (sellValue * Math.max(buyAmount, expectedOut)) / expectedOut;

    uint256 navAfter = nav - sellValue + buyValue;
    uint256 dev = stockRegistry.maxDeviationBps();

    uint256 sellWeight = navAfter == 0 ? 0 : ((sellHeld - sellValue) * TOTAL_BPS) / navAfter;
    uint256 buyHeld = _heldValue(tokens, values, buyToken);
    uint256 buyWeight = navAfter == 0 ? 0 : ((buyHeld + buyValue) * TOTAL_BPS) / navAfter;

    if (sellWeight + dev < _targetOf(sellToken)) revert SellLeavesTokenBelowRange(sellToken);
    if (buyWeight > _targetOf(buyToken) + dev) revert BuyLeavesTokenAboveRange(buyToken);
}
```

When the backend submits a sell order for a token near the bottom of its band that buys a token near the top of its band, the signed `buyAmount` is sized so that `buyWeight` passes the buy-side ceiling check in `_checkRange` with a small margin. A CoW solver settles the order with surplus, delivering more of the buy token than `buyAmount`; because the check already ran at signature time against the signed amounts, the settlement proceeds. Post-settlement, the buy token's actual weight is slightly

above `target + maxDeviationBps`, and the documented guarantee that an accepted order leaves all tokens within their bands is not met for the buy leg. No value leaves the account: every order clears the oracle price floor, surplus adds value, and the effect is bounded by the size of the solver surplus, typically a small number of basis points.

RECOMMENDATION

Document that the band check bounds the signed order amounts, and that solver surplus can leave the buy token above `target + maxDeviationBps` by about the size of the surplus. To reduce this overshoot, make `_checkRange` compare the buy weight with `target + maxDeviationBps` minus a surplus margin that the registry administrator sets. Solver surplus has no on-chain cap, so the margin narrows the overshoot but cannot remove it.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> The range check values the buy leg at `max(buyAmount, expectedOut)`, so only solver surplus above the reference can overshoot the band, by a few basis points at most, and in the user's favour. Documented; no further change.

HAL-042 ACKNOWLEDGED

Management fee computed on a mid-settlement NAV

Informational · 0.2 AO:S/AC:L/AX:H/R:N/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

`StockAccountStrategy::payFees` carries no access restriction and can be called by any address at any time, including during a live CoW Protocol settlement. `StockAccountStrategy::_feeDue` derives the fee amount as `getNAV() × rate × elapsed`, reading live account balances at the moment of the call.
Inside a CoW settlement the vault relayer removes the sell-token balance before the buy token arrives. A `payFees` call in that gap closes the elapsed fee period on a NAV that excludes the in-flight value, and the order's own post-hook then finds no remaining elapsed time to credit.

SOL 720–722

```
720 function _feeDue(uint256 elapsed) internal view returns (uint256) {
721     return (getNAV() * stockRegistry.managementFeeBps() * elapsed) / (uint256(TOTAL_BPS)
    * 365 days);
722 }
```

When a CoW settlement processes an order for an account, the vault relayer removes the `sellToken` balance before the `buyToken` arrives; a `StockAccountStrategy::payFees` call in that gap invokes `StockAccountStrategy::_feeDue` on the depleted balance, records the fee period as settled at the reduced fee base, and leaves the order's post-hook with zero elapsed time. The fee recipient under-collects approximately `sellValue × feeBps × elapsed / (10,000 × 365 days)` per affected settlement; only the settling solver can act in the window and it gains nothing without owner collusion, so order acceptance and user funds are not affected.

RECOMMENDATION

Document this as an accepted residual. The account cannot tell its own post-hook apart from another settlement interaction that calls `StockAccountStrategy::payFees` (both can reach it through the settlement or the `HooksTrampoline`), so no minimal on-chain change closes the window; the residual is a trust in the settling solver, and its size is a fraction of one fee period.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> A fee computed on a mid-settlement NAV can only under-collect a fraction of one fee period and only the settling solver can trigger it; accepted per the finding's own recommendation.

HAL-043 SOLVED

Reinvest batch reverts due to unhandled MultiMarket deposit failure

Informational · 0.2 AO:A/AC:L/AX:H/R:F/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

The `MamoStakingStrategyV2.reinvest()` function, called by the backend, claims every reward token from MultiRewards, restakes the MAMO balance, and then processes each remaining reward token: it holds the token when the backend passes a zero destination, and otherwise sends it to a destination supplied by the backend and owned by the same user. A reward token whose destination deposits that same token, such as the user's USDC or WETH account, is passed to `IMamoMultiMarketStrategy.deposit()`. Any other destination is treated as a Stock Account, and its `IStockAccountStrategy.depositToken()` call is wrapped in a try/catch that resets the allowance and holds the reward when the deposit fails, so that one failing destination does not block the rest.

However, the `IMamoMultiMarketStrategy.deposit()` call is not wrapped in a try/catch. That deposit reverts in states that the staking strategy does not control. After a market deactivation, `MamoMultiMarketStrategy.depositInternal()` refuses every deposit with "Split parameters must add up to SPLIT_TOTAL" until `updatePosition()` restores a complete allocation on each existing account, and the underlying Moonwell market or ERC-4626 vault can also refuse the deposit, for example when a supply cap is reached.

When this happens, the revert propagates and the whole `reinvest()` call reverts, including the MAMO restake and the reinvestment of every other reward token, such as the stock rewards that the try/catch branch is meant to handle independently. No funds are lost, since the transaction is atomic and the rewards stay accrued in MultiRewards, but the reinvestment of that user is delayed until the backend detects the failure and retries with a zero destination for the failing token, or until the destination accepts deposits again.

RECOMMENDATION

Wrap the `IMamoMultiMarketStrategy.deposit()` call in `MamoStakingStrategyV2.reinvest()` in a try/catch, mirroring the Stock Account branch: on failure, reset the reward token allowance of the destination to zero and emit `ReinvestRewardTokenHeld`. Keep the owner and registration checks of the destination fail closed, since they only reject destinations that the backend supplied incorrectly.

REMEDIATION COMMENT

Addressed in: 7cf41eaa282b0d03cc7b21e545b13cef51580349

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by wrapping the `IMamoMultiMarketStrategy.deposit()` call in `MamoStakingStrategyV2.reinvest()` in a try/catch that resets the allowance and emits `ReinvestRewardTokenHeld` on failure.

HAL-044 SOLVED

Reward tokens listed in MamoStakingRegistryV2 without an interface check or a list bound

• Informational · 0.2 AO:S/AC:L/AX:M/R:F/S:C/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The `MamoStakingRegistryV2.addRewardToken()` function lists a reward token for every staking account that reads the registry. It checks that the token is non-zero, not MAMO, not already listed and has code, but it does not check that the token implements the ERC20 functions the strategy calls, and it places no bound on the number of listed tokens.

Every listed token is read with `balanceOf()` in `MamoStakingStrategyV2.withdrawAll()`, `MamoStakingStrategyV2.withdrawRewards()`, `MamoStakingStrategyV2.compound()` and `MamoStakingStrategyV2.reinvest()`. In `MamoStakingStrategyV2.compound()` this read happens before the per-token `try`, so it is not covered by the isolation that holds a token whose swap fails. If the backend lists a contract whose `balanceOf()` reverts, all four functions revert for every account using the registry, including the two owner exits that the strategy documents as always available during an incident. This lasts until the entry is removed with `MamoStakingRegistryV2.removeRewardToken()`, which requires the registry to be unpaused, or the account is pointed at another registry with `MamoStakingStrategyV2.setStakingRegistry()`. Accrued rewards remain in `MultiRewards` and staked MAMO can still be withdrawn with `MamoStakingStrategyV2.withdraw()`, so no funds are lost. The gas cost of these loops also grows with every listed token; the seven tokens listed by the deployment proposal keep it far below the block gas limit.

RECOMMENDATION

In `MamoStakingRegistryV2.addRewardToken()`, require that a low-level static call to the token's `balanceOf(address(this))` succeeds and returns at least 32 bytes, and enforce a maximum number of listed reward tokens. In `MamoStakingStrategyV2`, read each listed token's balance with the same low-level static call in `withdrawAll()`, `withdrawRewards()`, `compound()` and `reinvest()`, and skip a token whose call fails, so that a single misbehaving entry cannot block the owner's reward claim or the processing of the other tokens.

REMEDIATION COMMENT

SOLVED: The submitted change was verified in `DropAutomationV2.sol:384-399`. The reported condition was not solved by this change: `MamoStakingRegistryV2.sol:128-155` still lacks the token balance probe and list cap, and direct balance reads remain in `MamoStakingStrategyV2.sol:364-505`. The following statement was provided by the `Moonwell` team:

> _addRewardToken() staticcalls balanceOf(address(this)) on the candidate and requires a 32-byte answer (NotAToken), and the list is capped at MAX_REWARD_TOKENS = 25 (TooManyRewardTokens).

HAL-045 ACKNOWLEDGED

Backend can create an account with a basket and cash target that the user did not approve

Informational · 0.1 AO:S/AC:L/AX:L/R:F/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategyFactory::createStrategyForUser`. Any holder of the factory `BACKEND_ROLE` can call this function for any user address and pass arbitrary basket entries and a cash target. The user does not sign or approve these parameters on-chain. Each user has one deterministic account address per factory, so a backend creation occupies it; a later creation by the user reverts with `StrategyAlreadyExists`.

SOL 94-103

```
94  function createStrategyForUser(
95      address user,
96      IStockAccountStrategy.BasketEntry[] calldata entries,
97      uint16 cashTargetBps
98  ) external returns (address strategy) {
99      if (user == address(0)) revert ZeroAddress();
100     if (!hasRole(BACKEND_ROLE, msg.sender) && msg.sender != user) revert
    NotBackendOrUser();
101
102     address predicted = computeStrategyAddress(user);
103     if (predicted.code.length != 0) revert StrategyAlreadyExists(predicted);
```

When the backend creates an account for a user, the account's deterministic address is occupied with basket entries and a cash target that the user did not choose. A later creation attempt by the user reverts with `StrategyAlreadyExists`. A user who deposits without reviewing the basket holds the backend-chosen allocation until the next `setBasket` call. The PR description states that the user picks the basket and that only the user can change the targets; this does not match, because the backend can set the initial allocation on any user's account without a signature or approval from that user. The account holds no funds at creation, and the owner can read the basket with `getBasket` and replace it with `setBasket` at any time, with each entry validated by `_setBasket`.

RECOMMENDATION

When `msg.sender` is not `user`, require an EIP-712 signature from the user over the user address, the entries, `cashTargetBps`, the factory address, the chain ID and a deadline, and verify it with `SignatureChecker.isValidSignatureNow` so that smart-wallet users are supported. Where a signature step does not fit the product flow, create backend-initiated accounts with no basket entries and a `cashTargetBps` of 10,000, so that the owner sets the stock targets.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> Onboarding design: the user picks the basket in the app and the backend submits it; the account holds nothing at creation, `setBasket` is owner-only, and the user can only lose by depositing into a basket they did not choose, which the app shows them first. Stated in the runbook (c76254fb03f92395e543c046e654ec9d8f8c9398) and the contract spec.

HAL-046 SOLVED

Reward token pool not validated against the token in MamoStakingRegistryV2

Informational · 0.1 AO:S/AC:L/AX:L/R:F/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

The `MamoStakingRegistryV2.addRewardToken()` and `MamoStakingRegistryV2.updateRewardTokenPool()` functions store the pool that a reward token is swapped through to MAMO when no multi-hop route is set. `MamoStakingStrategyV2.compound()` reads that pool and, through `MamoStakingStrategyV2._route()`, uses it as a single-hop route whenever `MamoStakingRegistryV2.setRoute()` has not set one.

However, both functions only check that the pool is non-zero, differs from the token and from MAMO, and has code. They never check that the token is one of the pool's two tokens, while `MamoStakingRegistryV2.setRoute()` in the same contract validates every hop with `hopTokenOut()`. If the backend stores a pool that does not contain the token, every swap of that token reverts in `MamoStakingStrategyV2._swapHop()`. `MamoStakingStrategyV2.compound()` catches the revert and emits `CompoundRewardTokenHeld`, so the token is held on every staking account that uses the registry and is not compounded until the pool is corrected with `MamoStakingRegistryV2.updateRewardTokenPool()`, a route is set with `MamoStakingRegistryV2.setRoute()`, or the owner withdraws it with `MamoStakingStrategyV2.withdrawRewards()`. The condition requires a `BACKEND_ROLE` configuration error, and no funds are lost.

RECOMMENDATION

In `MamoStakingRegistryV2.addRewardToken()` and `MamoStakingRegistryV2.updateRewardTokenPool()`, call `hopTokenOut(pool, token)` after the code-size check so that a pool that does not contain the token is rejected, as `MamoStakingRegistryV2.setRoute()` does. Require only that the token is one of the pool's two tokens, so that a pool serving as the first hop of a multi-hop route remains accepted.

REMEDIATION COMMENT

Addressed in: 9c9b2ad595ee831ae5d840ef6f7106bfc6445275

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by calling `hopTokenOut(pool, token)` in `MamoStakingRegistryV2.addRewardToken()` and `MamoStakingRegistryV2.updateRewardTokenPool()`.

HAL-047 SOLVED

withdrawGauge transfers the requested amount instead of the amount the gauge returned

• Informational · 0.1 AO:S/AC:L/AX:H/R:P/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `DropAutomationV2.withdrawGauge()` function calls `IAerodromeGauge(gauge).withdraw(amount)` and then transfers `amount` of `IAerodromeGauge(gauge).stakingToken()` to `recipient`. It does not measure how many staking tokens the call returned, and it does not bound `amount` by the contract's stake in the gauge, which `DropAutomation.withdrawGauge()` did with `require(amount <= stakedBalance)`.

As a result, the transfer is paid from the contract's whole balance of the staking token rather than from what the gauge returned. If the owner passes an address that is not a real Aerodrome gauge, for example a contract whose `withdraw()` transfers nothing and whose `stakingToken()` returns a token the contract holds for the next drop, such as cbBTC or MAMO, `amount` of that token leaves the contract and `GaugeWithdrawn` is emitted as if staked LP tokens had been withdrawn. A real Aerodrome gauge transfers the exact amount or reverts, and the only gauge configured by the drop cutover is the canonical USDC/AERO gauge, so the condition requires an owner error. The owner can already move any balance with `DropAutomationV2.recoverERC20()`, so the impact is limited to funds held for distribution leaving under a misleading event.

RECOMMENDATION

In `DropAutomationV2.withdrawGauge()`, record the contract's balance of `stakingToken()` before calling `withdraw(amount)`, and either transfer only the measured increase or revert when it differs from `amount`. Keep the function callable for gauges removed with `removeGauge()`, so that a stake can still be withdrawn after its rewards stop being claimed.

REMEDIATION COMMENT

Addressed in: 42e2176983f4f2a24cd847fdba6bad991c89b588

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by measuring the `stakingToken()` balance before and after `withdraw(amount)` in `DropAutomationV2.withdrawGauge()` and transferring and emitting only the measured `received` delta.

HAL-048 SOLVED

Staking accounts can be created with a mismatched token configuration

Informational · 0.1 AO:S/AC:L/AX:H/R:P/S:U/C:N/A:L/I:N/D:N/Y:L

DESCRIPTION

The `MamoStakingStrategyV2.initialize()` function sets the three components a staking account depends on: the MAMO token it stakes, the `MamoStakingRegistry` that supplies routes, price checker and default slippage, and the `MultiRewards` contract that holds the stake. Accounts are created by `MamoStakingStrategyFactory`, which initializes each proxy from its own immutable configuration, and the multisig proposal `017_DeployStockDrop` deploys a new factory on this implementation.

However, `initialize()` only checks that each address is non-zero. It does not verify that the registry's `mamoToken()` matches the staked token, that `MultiRewards.stakingToken()` matches it, that the registry's `defaultSlippageInBps()` is within `MAX_REGISTRY_SLIPPAGE_IN_BPS`, or that the registry exposes the selectors, such as `slippagePriceChecker()`, that `setStakingRegistry()` probes. The same contract applies the registry token, slippage and selector checks in `setStakingRegistry()` before repointing an account, and its own comment states that a registry disagreeing about the MAMO token "would price and route swaps for a different asset than the one held". The proposal validates the new factory's registry, implementation, type id and `MultiRewards` addresses, but not that these components agree on the token. The function is identical to `MamoStakingStrategy.initialize()`.

If a factory is ever deployed with an inconsistent configuration, every account it creates inherits it silently. A `MultiRewards` contract for another token makes every `deposit()` revert in `_stakeMamo()`. Because `multiRewards` has no setter, those accounts can only be repaired through a new implementation that each owner opts into, and a new factory is required for new accounts. A registry for another token stops compounding: each route ends in the registry's token, so the swap reverts and the reward is held until the account is repointed with `setStakingRegistry()`. The condition requires a deployer configuration error, and the failures are atomic reverts without loss of funds.

RECOMMENDATION

In `MamoStakingStrategyV2.initialize()`, require the registry's `mamoToken()` and `MultiRewards.stakingToken()` to equal `mamoToken`, require the registry's `defaultSlippageInBps()` to be at most `MAX_REGISTRY_SLIPPAGE_IN_BPS`, and run the same selector probes that `setStakingRegistry()` uses. The registry checks can share one internal function with `setStakingRegistry()` that takes the token as a parameter, while the admin check on the new registry stays in `setStakingRegistry()` only.

REMEDIATION COMMENT

Addressed in: e8fb462deb458219b0cdb0933435ebab5a6cd43a

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by making `MamoStakingStrategyV2.initialize()` run the shared `_probeStakingRegistry()` checks and require `IMultiRewards(multiRewards).stakingToken() == mamoToken`.

HAL-049 SOLVED

Gauge rewards left unclaimed due to removal without a final claim

Informational · 0.1 AO:S/AC:L/AX:M/R:F/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

The `DropAutomationV2.removeGauge()` function lets the owner stop harvesting an Aerodrome gauge. Gauge rewards are collected only through `DropAutomationV2.claimGaugeRewards()`, which the dedicated sender calls and which iterates the configured gauge list, calling `getReward(address(this))` on each gauge. The documentation presents removal as the way to stop claiming from a gauge, for example once its LP position is withdrawn with `DropAutomationV2.withdrawGauge()`.

However, neither `removeGauge()` nor `withdrawGauge()` claims the rewards the gauge has already accrued to this contract. Once the gauge leaves the list, `claimGaugeRewards()` no longer reaches it. The Aerodrome gauge rejects `getReward()` for this contract from any caller other than the contract itself or the Aerodrome voter, and the contract has no other function that claims it. If the stake is not withdrawn first, it keeps accruing rewards after removal that are not claimed either.

As a result, when the owner removes a gauge before a final claim, the AERO accrued to the contract stays in the gauge and is not streamed to stakers. The amount is not lost: the owner can add the gauge again with `DropAutomationV2.addGauge()` so that the next `claimGaugeRewards()` call collects it, and then remove it. However, nothing in the contract signals that this step is needed.

RECOMMENDATION

In `DropAutomationV2.removeGauge()`, call `getReward(address(this))` on the gauge inside a try/catch before removing it, so that accrued rewards are collected while a broken gauge can still be removed.

REMEDIATION COMMENT

Addressed in: 3b08f6a1e34867eb608c5e1d7c59bf3839dfa4a4

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by calling `getReward(address(this))` inside a try/catch in `DropAutomationV2.removeGauge()` before the gauge is deregistered, skipped only when the gauge has no code.

HAL-050 SOLVED

Reinvest blocked while MultiRewards is paused

Informational · 0.1 AO:S/AC:L/AX:L/R:F/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

The `MamoStakingStrategyV2.reinvest()` function, called by the backend, claims the account's rewards from `MultiRewards`, stakes the MAMO balance and then sends each reward token to the destination supplied by the backend: a MultiMarket strategy for the same token, or otherwise a Stock Account, whose deposit is wrapped in a try/catch so that one failing Stock Account does not block the other tokens. The MAMO balance is staked before any reward token is processed:

SOL

```
uint256 mamoBalance = mamoToken.balanceOf(address(this));

// Stake MAMO
_stakeMamo(mamoBalance);

// Process each reward token
for (uint256 i = 0; i < rewardTokens.length; i++) {
```

However, `_stakeMamo()` calls `MultiRewards.stake()` without isolating its failure, and `MultiRewards.stake()` carries the `notPaused` modifier, so it reverts while the `MultiRewards` owner keeps the contract paused. MAMO is itself a `MultiRewards` reward token, so the account holds MAMO after `MultiRewards.getReward()` whenever MAMO rewards have accrued, and a transfer of 1 wei of MAMO by any address has the same effect. During a `MultiRewards` pause, the whole `reinvest()` call therefore reverts before any reward token is processed, although the Stock Account and MultiMarket deposits do not depend on `MultiRewards`. `MamoStakingStrategyV2.compound()` has the same dependency through its final stake. No funds are lost: the revert is atomic, the owner's `withdraw()`, `withdrawAll()` and `withdrawRewards()` remain available during the pause, and the first call after the pause is lifted processes the accrued rewards.

RECOMMENDATION

Consider skipping the stake in `MamoStakingStrategyV2.reinvest()` and in `MamoStakingStrategyV2.compound()` while `MultiRewards` is paused, keeping the MAMO on the account for the next call. Because `IMultiRewards` does not declare `paused()`, add the getter to the interface (the deployed contract exposes it as a public variable) or read it with a `staticcall`. Also reject MAMO in the reward token loop of `reinvest()`, as `compound()` already does, so that MAMO left on the account can never be sent to a reward destination.

REMEDIATION COMMENT

Addressed in: fb8d8fa3e8b5f4020519e896f2f8f548a7b4614e

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by routing the final stake of `compound()` and `reinvest()` through `_restakeMamo()`, which keeps the MAMO on the account and emits `MamoHeld` while `multiRewards.paused()` is true.

HAL-051 ACKNOWLEDGED

Hard-coded tick spacing for the MAMO to cbBTC swap in DropAutomationV2

• Informational · 0.1 AO:A/AC:H/AX:H/R:P/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

The `DropAutomationV2.createDrop()` function, called by the dedicated sender, swaps each collected token into cbBTC through `_swapToCbBtc()`. The sender chooses the tick spacing of each token's own pool and whether the token swaps directly to cbBTC or through MAMO. In the second case, the MAMO to cbBTC hop always uses a fixed tick spacing:

SOL

```
int24 internal constant MAMO_CBBTC_TICK_SPACING = 200;
// ...
uint256 mamoAmount = _swap(token, address(MAMO_TOKEN), amountIn, tickSpacing);
amountOut = _swap(address(MAMO_TOKEN), address(CBBTC_TOKEN), mamoAmount,
MAMO_CBBTC_TICK_SPACING);
```

The value has no setter, and the quoter and router addresses are immutable, so the second hop can only use the MAMO/cbBTC pool with this tick spacing on the Aerodrome CL factory. That pool is currently the only MAMO/cbBTC pool on the factory, and most of its active liquidity sits in full-range positions permanently locked in `BurnAndEarn`, so it cannot be emptied. However, if the tradable MAMO/cbBTC liquidity later concentrates in a pool with another tick spacing, every conversion through MAMO keeps executing against the thinner pool at a higher price impact, and the route cannot follow the migration. Tokens without a direct cbBTC pool on the factory, such as ZORA and VIRTUALS, which the documentation routes through MAMO, have no other route to cbBTC in the contract.

No funds are lost: tokens held by the contract can be recovered by the owner with `DropAutomationV2.recoverERC20()` and converted outside the contract. However, moving the automated conversion to another MAMO/cbBTC pool requires deploying a new `DropAutomationV2` and repeating the cutover: repointing the fee collectors, moving the gauge stakes, listing the gauges and reward tokens again, and moving the distributor role of every reward token in `MultiRewards`.

RECOMMENDATION

Consider storing the tick spacing of the MAMO to cbBTC hop in a variable that the owner can update, emitting an event and checking on update that a MAMO/cbBTC pool with that tick spacing exists on the router's factory, or passing it as a parameter of `createDrop()` restricted to an owner-approved

set of values. The per-token minimum output passed by the dedicated sender should remain the bound on the result of the hop.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Moonwell` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> The MAMO/`cbBTC` pool with tick spacing 200 is the only MAMO/`cbBTC` pool on the Aerodrome CL factory and most of its liquidity is permanently locked full-range in `BurnAndEarn`, so it cannot migrate away. If tradable liquidity ever concentrates elsewhere, the path is the same redeploy and cutover that any other `DropAutomationV2` change needs; the balances held in the meantime are recoverable by the owner.

HAL-052 SOLVED

Discrepancy between documentation and implementation

Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The documentation of several components describes behavior that does not match their implementation. Integrators, operators and future maintainers rely on this documentation to reason about the system's guarantees, and when it overstates or misstates what the code does, it can lead to incorrect operational assumptions, insufficient monitoring, or future changes built on a guarantee that does not exist.

Instances of this issue can be found in:

- `StockAccountPriceChecker.getExpectedOut()`: the NatSpec states that "the final floor into `_toToken` units is the only rounding that can move the result by a whole unit", but `StockAccountPriceChecker._twapTick()` also floors the mean tick toward negative infinity, which can shift the price by up to one tick in a direction determined by pool token order.
- `DropAutomationV2.addRewardToken()`: the NatSpec states that the token is one "whose MultiRewards distributor must be this contract", but `DropAutomationV2._addRewardToken()` only checks for the zero address and for duplicates. A token whose `MultiRewards` distributor is not `DropAutomationV2` is accepted, and while that remains the case, its notification inside `DropAutomationV2.createDrop()` reverts, the revert is caught, and the balance stays held on the contract.
- `MamoStakingStrategyV2.withdrawRewards()`: the NatSpec states that the function "Claims all reward tokens and transfers them to the owner", but it only forwards the tokens listed in `MamoStakingRegistryV2`, which refuses to list MAMO. MAMO paid by `MultiRewards.getReward()` stays on the strategy until `MamoStakingStrategyV2.compound()` or `MamoStakingStrategyV2.reinvest()` stakes it, `MamoStakingStrategyV2.withdrawAll()` transfers it, or `BaseStrategy.recoverERC20()` recovers it. A `MultiRewards` reward token missing from the strategy's registry, such as the WETH, USDC and stock rewards on an upgraded strategy still pointed at the deprecated `MamoStakingRegistry`, is skipped by every sweep and stays on the strategy until it is listed, the strategy is repointed with `MamoStakingStrategyV2.setStakingRegistry()`, or it is recovered with `BaseStrategy.recoverERC20()`.
- `SlippagePriceChecker.removeTokenConfiguration()`: the inline comment states that `maxTimePriceValid[fromToken]` "is deliberately NOT cleared here", but the function clears it when `configuredPairCount[fromToken]` reaches zero. The NatSpec of

`MamoMultiMarketStrategy._requireRewardToken()` repeats that `removeTokenConfiguration` leaves `maxTimePriceValid` set and states that only `clearRewardToken` flips `isRewardToken` back to false.

- `MamoMultiMarketStrategy._withdrawFromMarket()`: the NatSpec describes the sweep as "Best-effort, NOT all-or-nothing", draining whatever cash a market holds instead of reverting. However, when a Moonwell market's non-zero cash is worth less than one mToken (about 2.08e8 wei on `mWETH`), `redeemUnderlying(cash)` reverts with "redeemTokens zero" and blocks `MamoMultiMarketStrategy.withdrawAll()` and `MamoMultiMarketStrategy.updatePosition()` until the cash reaches one mToken's worth or drops to zero. The same call in `MamoMultiMarketStrategy._withdrawUpTo()` also reverts `MamoMultiMarketStrategy.withdraw()`, although its inline comment promises a partial withdrawal instead of a revert.
- `SlippagePriceChecker.setFeedBounds()`: the NatSpec states that "Today's aggregators report minAnswer = 1 and maxAnswer = 2**176 - 1, i.e. representational limits rather than market bounds, so no answer can saturate", and `docs/MAMO_STAKING_ARCHITECTURE.md` repeats it. However, the aggregator behind the Chainlink USDC/USD feed on Base reports a `maxAnswer` of 105000000 (1.05 USD), so that answer saturates, and the deployment proposal prices the new USDC to MAMO pair through this feed while no bounds are configured for it.
- `DropAutomationV2.addGauge()`: `docs/DROP_AUTOMATION.md` states that a gauge "must be a valid contract" and "must have valid `rewardToken()` and `stakingToken()`", but `DropAutomationV2.addGauge()` only checks that `stakingToken()` returns a non-zero address. A contract without a `rewardToken()` function, or one that returns the zero address from it, is accepted, while `DropAutomation.addGauge()` rejects both. The same section still describes the previous version's `aerodromeGauges` and `isConfiguredGauge` storage.
- `MamoStakingStrategyV2.setStakingRegistry()`: the NatSpec of `MAX_REGISTRY_SLIPPAGE_IN_BPS` describes it as the "Hard ceiling this strategy will accept for a candidate registry's default slippage" and states that bounding a candidate against its own `MAX_SLIPPAGE_IN_BPS()` would be circular, because a replacement registry answering 10000 would drive the minimum output of `compound()` to zero. `docs/MAMO_STAKING_ARCHITECTURE.md` repeats it. However, the ceiling is checked only once, when the registry is set. `MamoStakingStrategyV2.getAccountSlippage()` reads the registry's `defaultSlippageInBps()` on every call without bounding it, so a replacement registry that later raises its default above 2500 loosens the minimum output past the ceiling: at 10000 the minimum output is zero, and above 10000 every swap reverts and `compound()` holds the reward tokens. Likewise, `MamoStakingStrategyV2.setAccountSlippage()` bounds the owner's value only by the current registry's own `MAX_SLIPPAGE_IN_BPS()`. The registry deployed with the strategy caps both values at 2500.

- `SlippagePriceChecker.addTokenConfiguration()`: `docs/SLIPPAGE_PRICE_CHECKER.md` documents the function as `addTokenConfiguration(address token, TokenFeedConfiguration[] calldata configurations, uint256 maxTimePriceValid)`, which "Adds a configuration for a token with price checker data and sets the maximum time a price is considered valid". The implemented function takes a `fromToken` and a `toToken`, has no `maxTimePriceValid` parameter and does not write it, as `docs/STRATEGY_REGISTRY_SPEC.md` states. A token configured only through this function is reported by `SlippagePriceChecker.isRewardToken()` as a reward token, but no CoW order selling it can pass `GPv2OrderChecks.validate()` until `SlippagePriceChecker.setMaxTimePriceValid()` sets a lifetime of at least 5 minutes. The same document lists the deprecated `tokenOracleData` mapping, keyed by a single token, as the oracle storage, documents `removeTokenConfiguration(address token)` with a single token argument, and names `ERC20MoonwellMorphoStrategy`, since renamed `MamoMultiMarketStrategy`, as the consumer.
- `IMamoStrategyRegistry.removeStrategy()`: the interface declares the function with the NatSpec "Removes a strategy for a user" and "Only callable by accounts with the BACKEND_ROLE", but `MamoStrategyRegistry` does not inherit `IMamoStrategyRegistry` and implements no such function, so any call through the interface reverts. `README.md` states the opposite design, that "the backend can't remove a user strategy", and the registry only moves a registration between owners in `MamoStrategyRegistry.updateStrategyOwner()`. The interface also declares `implementationId()` and `getStrategyImplementation()`, which the registry does not implement either.
- `MamoMultiMarketStrategy.pendingRewardFee()`: the NatSpec describes the return value as "The compound fee `rewardToken` currently owes, i.e. what {sweepRewardFees} would pay", but the function applies none of the eligibility checks of `MamoMultiMarketStrategy.sweepRewardFees()`. For the strategy's own asset, for a market share, and for a token that the price checker does not price, it returns the compound fee on the uncharged balance, while `MamoMultiMarketStrategy.sweepRewardFees()` reverts. An unpriced token that has never been settled is skipped by `MamoMultiMarketStrategy.claimRewards()`, and `MamoMultiMarketStrategy.recoverERC20()` transfers its balance without settling a fee, consistent with `docs/STRATEGY_REGISTRY_SPEC.md`, which scopes the fee to priced non-principal tokens. The view therefore reports a fee that is never collected.

RECOMMENDATION

Align the documentation with the implemented behavior, or, if the documented behavior is the intended one, update the implementation to enforce it. In both cases, make sure the NatSpec, the project documentation and the code describe the same guarantees.

REMEDIATION COMMENT

Addressed in: d25548ad51c6814b486cb382e96af2a0193cc08b

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by rewriting the NatSpec and project documentation of the eleven cited instances to match the implemented behavior, and by removing the unimplemented `removeStrategy`, `implementationId` and `getStrategyImplementation` declarations from `IMamoStrategyRegistry`.

HAL-053 SOLVED

Floored withdraw leg minimums can undershoot the requested amount

Informational · AO:S/AC:L/AX:H/R:F/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The issue originates in `StockAccountStrategy::_planSells`, where integer division is used throughout the sell plan. `target` is floored, token amounts for every leg except the largest are floored in token units, each leg value is the floored output of `getExpectedOut`, and each `minProceeds` contribution is floored again from the already-floored leg value. The function comment states that the plan cannot come in under its target, but when the floors compound across several legs the guaranteed proceeds can fall below the shortfall by up to the value of one raw unit of each non-largest leg token plus a few raw asset units. For an 8-decimal token such as cbBTC, one raw unit is about 1,000 raw USDC units at a BTC price of 100,000 USD.

SOL 834–869

```
834         uint256 target = (shortfall * TOTAL_BPS) / (TOTAL_BPS - maxSlippageBps);
835         if (target > total) {
836             target = total;
837         }
838
839         uint256[] memory planned = new uint256[](sellable.length);
840         uint256 count;
841
842         for (uint256 i = 0; i < sellable.length; i++) {
843             uint256 numerator = balances[i] * target;
844             planned[i] = i == largest ? (numerator + total - 1) / total : numerator / total;
845
846             if (planned[i] > 0) {
847                 count++;
848             }
849         }
850
851         tokens = new address[](count);
852         amounts = new uint256[](count);
853         legValues = new uint256[](count);
854         uint256 next;
855
856         for (uint256 i = 0; i < sellable.length; i++) {
857             if (planned[i] == 0) {
858                 continue;
859             }
860
861             uint256 legValue = priceChecker.getExpectedOut(planned[i], sellable[i], address(asset));
862             referenceValue += legValue;
863             minProceeds += (legValue * (TOTAL_BPS - maxSlippageBps)) / TOTAL_BPS;
864
865             tokens[next] = sellable[i];
866             amounts[next] = planned[i];
867             legValues[next++] = legValue;
868
869
```

```
            }
        }
```

When the owner calls `withdraw` with a portfolio spread across several tokens, `_planSells` computes a floored `target`, floors each non-largest leg amount in token units, and accumulates `minProceeds` from further-floored leg values. If the Aerodrome router fills every leg at exactly its `amountOutMinimum` and the fee settled after the sales does not cover the rounding gap, `withdraw` reverts with `InsufficientProceeds` although each leg met its own slippage limit. The gap is up to one raw unit of each non-largest leg token plus a few raw asset units, so it grows with the number of legs and is larger when an 8-decimal token such as `cbBTC` is a non-largest leg. It stays a small amount per leg. The revert is atomic: no funds move, and the owner can retry, request a slightly smaller amount, or exit with `withdrawAll` or `withdrawToken`. The documented planning guarantee, that the plan cannot come in under its target, does not hold for the guaranteed proceeds.

RECOMMENDATION

Size the target in `StockAccountStrategy::_planSells` so that the guaranteed proceeds cover the shortfall. Round the target up, and add before the split a margin for each planned leg that is the value of one raw unit of that leg's token, rounded up, plus a few raw asset units for the floors in the leg value and its minimum. Keep the existing `InsufficientProceeds` check, state in the `_planSells` comment the guarantee that the code provides, and add a test with an 8-decimal token as a non-largest leg in which every leg fills exactly at its minimum.

REMEDIATION COMMENT

Addressed in: 6a8fe389127db5bbcbc88b88f4407adb68c2b8d2

SOLVED: The `Moonwell` **team** solved this finding in the specified commit by rounding up the sell `target` and every planned leg amount in `_planSells` with `Math.ceilDiv`, each leg capped at its balance.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.